

THIẾT KẾ CƠ SỞ DỮ LIỆU & SƠ ĐỒ THỰC THỂ – LIÊN KẾT (ERD)

Multi-Warehouse E-Commerce & Inventory Control System

Đề tài	Topic 3 — Retail & Supply Chain
Môn học	Software Engineering Final Project — Course ID 502045
Nhóm	5 thành viên (P1 – P5)
Hệ quản trị CSDL	PostgreSQL 16 + extension pgvector, pg_trgm, pgcrypto
ORM	Prisma 7.10.0
Quy mô schema	18 bảng · 179 cột · 10 kiểu ENUM · 25 quan hệ · 21 CHECK · 62 index
Nguồn yêu cầu	Bản đặc tả v2 (ISO/IEC/IEEE 29148): 29 FR · 26 NFR · 7 CON · 4 IF
Trạng thái	Đã kiểm chứng thực thi trên PostgreSQL 16.15
Phiên bản tài liệu	v1.0

MỤC LỤC

MỤC LỤC.....	2
1. GIỚI THIỆU & PHẠM VI THIẾT KẾ.....	4
1.1. Mục đích tài liệu	4
1.2. Bài toán cốt lõi mà CSDL phải giải	4
1.3. Phạm vi.....	4
2. TÁM NGUYÊN TẮC THIẾT KẾ ĐƯỢC ÁP DỤNG	4
3. XUNG ĐỘT GIỮA CÁC TÀI LIỆU & QUYẾT ĐỊNH XỬ LÝ.....	6
4. CÁC KIỂU ENUM	7
5. ĐẶC TẢ CHI TIẾT 18 BẢNG.....	9
5.1. Nhóm A — Identity & Access (sở hữu: P4)	9
Bảng 1. users — Người dùng hệ thống	9
Bảng 2. refresh_tokens — Refresh token	9
5.2. Nhóm B — Catalog (sở hữu: P4)	10
Bảng 3. categories — Danh mục sản phẩm.....	10
Bảng 4. products — Sản phẩm (catalog gốc).....	10
5.3. Nhóm C — Warehouse & Inventory (sở hữu: P4) — LỖI KỸ THUẬT	11
Bảng 5. warehouses — Kho hàng khu vực.....	11
Bảng 6. inventory — Tồn kho (BẢNG TRUNG TÂM)	11
Bảng 7. inventory_audit_log — Nhật ký biến động tồn kho	12
5.4. Nhóm D — Cart (sở hữu: P1) — tùy chọn	13
Bảng 8. carts — Giỏ hàng.....	13
Bảng 9. cart_items — Dòng sản phẩm trong giỏ	13
5.5. Nhóm E — Order & Reservation (sở hữu: P3) — LỖI KỸ THUẬT.....	14
Bảng 10. orders — Đơn hàng.....	14
Bảng 11. order_items — Dòng sản phẩm trong đơn.....	14
Bảng 12. stock_reservations — Giữ chỗ tồn kho (CHỐNG OVERSELL)	15
Bảng 13. order_status_history — Lịch sử chuyển trạng thái đơn.....	15
5.6. Nhóm F — Payment (sở hữu: P3).....	17
Bảng 14. payments — Giao dịch thanh toán (mô phỏng)	17
5.7. Nhóm G — Notification (sở hữu: P5).....	18
Bảng 15. stock_alerts — Cảnh báo tồn kho thấp.....	18
5.8. Nhóm H — AI Chatbot RAG (sở hữu: P5).....	19
Bảng 16. kb_documents — Tài liệu tri thức (nguồn RAG tĩnh)	19
Bảng 17. chat_sessions — Phiên hội thoại Chatbot.....	19
Bảng 18. chat_messages — Tin nhắn hội thoại	19
6. QUAN HỆ GIỮA CÁC BẢNG	21
6.1. Quan hệ N–N cốt lõi.....	22
6.2. Quan hệ tự tham chiếu.....	22

7. RÀNG BUỘC TOÀN VỆ & CHIẾN LƯỢC INDEX	23
7.1. Các ràng buộc chống oversell ở tầng CSDL	23
7.2. Các index quan trọng và lý do tồn tại	23
8. SƠ ĐỒ THỰC THỂ – LIÊN KẾT (ERD)	24
8.1. ERD tổng quan toàn hệ thống	24
8.2. ERD lỗi — luồng chống oversell	25
8.3. ERD module AI Chatbot (RAG)	26
9. KẾT QUẢ KIỂM CHỨNG THỰC TẾ	27
9.1. Kết quả tạo schema	27
9.2. Kiểm thử tình huống hai khách tranh sản phẩm cuối cùng	27
9.3. Kiểm thử cron job giải phóng reservation hết hạn	28
9.4. Một ràng buộc đã bị loại bỏ sau khi kiểm thử	28
10. MÃ NGUỒN DDL (PostgreSQL)	29
10.1. Script tạo toàn bộ schema	29
10.2. Hàm giữ chỗ tồn kho mẫu (bản cài đặt đúng)	34
11. SCHEMA PRISMA	36
12. MA TRẬN TRUY VẾT YÊU CẦU → BẢNG DỮ LIỆU	41
13. NHỮNG ĐIỂM NHÓM CẦN CHỐT TRƯỚC KHI CODE	42
13.1. Việc cần làm tiếp theo	42

(Mở bằng Microsoft Word, bấm chuột phải vào mục lục → Update Field để hiện số trang.)

1. GIỚI THIỆU & PHẠM VI THIẾT KẾ

1.1. Mục đích tài liệu

Tài liệu này là bản thiết kế cơ sở dữ liệu hoàn chỉnh cho hệ thống Multi-Warehouse E-Commerce & Inventory Control System, đồng thời là bằng chứng số bắt buộc của Phase 2 — System Design theo Student Handbook (mục CON-03: thư mục /docs phải có ERD cho tồn kho đa kho).

Thiết kế được xây dựng bám sát Bản đặc tả yêu cầu phần mềm phiên bản 2 (chuẩn ISO/IEC/IEEE 29148) — tài liệu tự xác định là “bản chốt requirement dùng bàn giao cho các phần ERD / Sequence / Timeline / Code”. Mục 7 của bản đặc tả nêu rõ: “ERD phải phản ánh các entity/data cần thiết cho FR đã chốt, đặc biệt Product, Warehouse, Inventory, Order, Reservation, Payment và các dữ liệu phục vụ Notification/Chatbot”.

1.2. Bài toán cốt lõi mà CSDL phải giải

Toàn bộ thiết kế xoay quanh một yêu cầu duy nhất: NFR-04.1 — hệ thống phải có 0 trường hợp oversell. Khi hàng trăm khách cùng lúc mua sản phẩm cuối cùng trong kho, cơ sở dữ liệu phải bảo đảm tối đa đúng số lượng tồn kho được bán ra, không một đơn vị nào hơn.

Từ ràng buộc này, ba quyết định kiến trúc được suy ra và chi phối toàn bộ schema:

- Tách hoàn toàn số lượng tồn kho ra khỏi bảng sản phẩm — số lượng chỉ tồn tại trong bảng inventory theo từng cặp (sản phẩm, kho).
- Không trừ kho trực tiếp khi đặt hàng — dùng cơ chế giữ chỗ (reservation) hai pha có thời hạn, chỉ trừ kho thật khi thanh toán thành công.
- Đặt lớp phòng thủ ở tầng CSDL (CHECK constraint, UNIQUE index, trigger) chứ không chỉ tin vào code ứng dụng.

1.3. Phạm vi

Trong phạm vi	Ngoài phạm vi
Schema đầy đủ 18 bảng cho 29 FR đã chốt	Dữ liệu thẻ thanh toán thật (bị cấm bởi CON-05, NFR-02.3)
Ràng buộc toàn vẹn, index, trigger	Phân vùng (partitioning), sharding — chưa cần ở quy mô đồ án
Ba biến thể ERD (tổng quan / lỗi / chatbot)	Replication, backup policy — thuộc phạm vi vận hành
DDL PostgreSQL và schema Prisma	Dữ liệu seed chi tiết — thuộc file Seed.ts của P4

2. TÁM NGUYÊN TẮC THIẾT KẾ ĐƯỢC ÁP DỤNG

Mã	Nguyên tắc	Diễn giải & lý do	Truy vết
NT-01	Tách số lượng khỏi sản phẩm	Bảng products tuyệt đối không có cột số lượng. Mọi số lượng nằm ở inventory theo cặp (product_id, warehouse_id). Đây là điều kiện tiên quyết để hỗ trợ đa kho — nếu để số lượng trong products thì không thể biết hàng nằm ở kho nào.	FR-INV-01
NT-02	Giữ chỗ hai pha thay vì trừ kho trực tiếp	Đặt hàng chỉ tăng reserved_stock và tạo bản ghi HOLDING có TTL. Chỉ khi thanh toán SUCCESS mới trừ stock thật. Nhờ vậy, khách bỏ ngang thanh toán không làm mất hàng vĩnh viễn.	FR-ORD-02, FR-PAY-02
NT-03	Khóa bi quan theo thứ tự ổn định	inventory.id dùng BIGSERIAL (số tăng dần) thay vì UUID. Lý do: mọi transaction luôn SELECT ... FOR UPDATE theo thứ tự inventory_id tăng dần, nên không bao giờ xảy ra deadlock chéo khi một đơn có nhiều sản phẩm. UUID ngẫu nhiên không cho thứ tự ổn định này.	FR-ORD-02, NFR-04.1

Mã	Nguyên tắc	Diễn giải & lý do	Truy vết
NT-04	Snapshot dữ liệu tại thời điểm giao dịch	order_items lưu bản sao tên sản phẩm, SKU, đơn giá, giảm giá. Khi Admin đổi giá sản phẩm (FR-ADM-01), đơn hàng cũ vẫn giữ nguyên giá đã bán — đúng yêu cầu “Đơn hàng đã tạo giữ nguyên giá tại thời điểm đặt”.	FR-ADM-01, FR-CART-03
NT-05	Không dùng số thực cho tiền tệ	Mọi cột tiền dùng NUMERIC(14,2), ánh xạ sang Decimal trong Prisma. Tuyệt đối không dùng FLOAT/DOUBLE vì sai số dấu phẩy động sẽ làm lệch tổng tiền — đặc tả yêu cầu grandTotal “không sai số dấu phẩy động”.	FR-CART-03
NT-06	Idempotency cưỡng chế ở tầng CSDL	payments.transaction_ref có UNIQUE constraint, cộng thêm partial unique index bảo đảm mỗi đơn chỉ có tối đa một giao dịch SUCCESS. Callback lập bị chặn bởi database chứ không phụ thuộc vào việc lập trình viên nhớ kiểm tra.	FR-PAY-02, NFR-04.2
NT-07	Mọi biến động đều để lại dấu vết	inventory_audit_log ghi giá trị trước/sau mỗi lần đổi kho; order_status_history ghi mọi lần chuyển trạng thái; chat_messages lưu retrieved_context. Đây là dữ liệu để đối soát khi kiểm thử NFR-03.1, NFR-03.2 và chấm điểm chatbot NFR-04.4.	NFR-03.x, NFR-04.4
NT-08	Ranh giới module phản ánh trong sở hữu bảng	Mỗi bảng có đúng một module sở hữu (ghi), các module khác chỉ đọc qua service interface. Bảng inventory do P4 sở hữu; P3 muốn khóa kho phải gọi InventoryService chứ không tự viết SQL — đúng NFR-02.4 và NFR-06.1.	NFR-02.4, NFR-06.1

3. XUNG ĐỘT GIỮA CÁC TÀI LIỆU & QUYẾT ĐỊNH XỬ LÝ

Khi đối chiếu ba nguồn — bản Kế hoạch triển khai Topic 3, bản Phân công công việc v2 và Bản đặc tả v2 — phát hiện chín điểm không khớp. Vì Bản đặc tả v2 tự khẳng định là bản chốt bàn giao cho ERD, các xung đột được xử lý theo hướng ưu tiên bản đặc tả. Toàn bộ được liệt kê dưới đây để nhóm rà soát lại.

Mã	Vấn đề	Kế hoạch / nguồn A	Đặc tả / nguồn B	Quyết định áp dụng
XĐ-01	Số vai trò người dùng	Kế hoạch Topic 3: chỉ 2 vai trò (customer / admin)	Đặc tả FR-AUTH-02: 3 vai trò CUSTOMER, WAREHOUSE_STAFF, ADMIN	Dùng 3 vai trò. Thiếu WAREHOUSE_STAFF thì không ai thực hiện được FR-ORD-05 (cập nhật trạng thái giao hàng).
XĐ-02	Tập trạng thái đơn hàng	Kế hoạch: pending / paid / reserved / fulfilled / cancelled	Đặc tả: PENDING_PAYMENT → CONFIRMED → SHIPPING → DELIVERED / CANCELLED	Dùng bộ của đặc tả. Bộ cũ trộn lẫn trạng thái thanh toán với trạng thái giao hàng và thiếu bước SHIPPING mà chatbot cần tra cứu.
XĐ-03	Trạng thái reservation	Kế hoạch: holding / committed / released (3 giá trị)	Đặc tả FR-ORD-04: cần phân biệt hết hạn tự động	Bổ sung giá trị EXPIRED. Không có nó thì không phân biệt được “hệ thống tự thu hồi” với “người dùng chủ động hủy”, gây khó khi đối soát NFR-04.3.
XĐ-04	Tên cột tồn kho	Kế hoạch: quantity_on_hand, quantity_reserved, low_stock_threshold	Đặc tả: stock, reserved_stock, threshold, availableStock	Dùng tên của đặc tả vì các Acceptance Criteria và kịch bản kiểm thử NFR đều viết theo tên này. Cần thông báo cho cả nhóm để API contract dùng thống nhất.
XĐ-05	Lịch sử trạng thái đơn	Kế hoạch: không có bảng nào	Đặc tả FR-ORD-05: bắt buộc ghi order_status_history	Thêm bảng order_status_history. Đây là bảng mới hoàn toàn so với danh sách 12 bảng trong kế hoạch.
XĐ-06	Refresh token	Kế hoạch chốt “JWT + Refresh Token” nhưng không có bảng lưu	Đặc tả FR-AUTH-01 chỉ nói JWT access token	Thêm bảng refresh_tokens (lưu dạng băm). Nếu nhóm quyết định bỏ refresh token thì xóa bảng này, không ảnh hưởng bảng khác.
XĐ-07	Ngưỡng cảnh báo low-stock	Kế hoạch: đặt ở inventory.low_stock_threshold	Đặc tả mục 9: còn để ngỏ — theo kho hay theo cặp sản phẩm-kho?	Thiết kế hỗ trợ CẢ HAI: warehouses.default_low_stock_threshold làm mặc định cấp kho, inventory.threshold cho phép ghi đè cho từng cặp. Nhóm chọn cách nào cũng chạy được mà không đổi schema.
XĐ-08	Nơi lưu giỏ hàng	Kế hoạch P1: có thể bỏ backend, chỉ dùng Zustand	Đặc tả FR-CART-01: nhóm phải tự chốt	Vẫn thiết kế bảng carts và cart_items nhưng đánh dấu TỰ CHỌN. Nếu chốt client-side, xóa hai bảng này — không bảng nào khác phụ thuộc vào chúng.
XĐ-09	Nguồn sinh vector embedding	Kế hoạch: dùng Anthropic Claude API cho toàn bộ AI	Thực tế kỹ thuật: Anthropic KHÔNG cung cấp API embedding	Claude chỉ sinh câu trả lời (FR-BOT-02). Việc sinh vector cho kb_documents.embedding phải dùng nhà cung cấp khác (Voyage AI — chính Anthropic khuyến nghị, hoặc OpenAI embeddings, hoặc mô hình chạy cục bộ). Số chiều vector(1024) phải khớp mô hình đã chọn.

Điểm cần nhóm quyết ngay — XĐ-09

Kế hoạch đang giả định Claude API làm được cả embedding lẫn sinh câu trả lời. Thực tế Anthropic không có endpoint embedding, nên P5 sẽ bị chặn ngay khi bắt tay làm RAG ở Sprint 2.

Cần chốt nhà cung cấp embedding trong Sprint 1 vì số chiều vector quyết định kiểu cột trong schema: Voyage AI voyage-3 là 1024 chiều, OpenAI text-embedding-3-small là 1536 chiều. Đổi số chiều sau khi đã nạp dữ liệu đồng nghĩa phải tạo lại bảng và vector hóa lại toàn bộ tài liệu.

4. CÁC KIỂU ENUM

PostgreSQL ENUM được dùng thay cho cột VARCHAR tự do vì ba lý do: CSDL tự chặn giá trị sai ngay tại tầng dữ liệu, Prisma sinh ra kiểu TypeScript tương ứng giúp bắt lỗi lúc biên dịch, và bộ giá trị trở thành tài liệu sống cho cả nhóm.

Tên ENUM	Các giá trị	Mô tả & ghi chú	Tru y vết
<code>user_role</code>	CUSTOMER • WAREHOUSE_STAFF • ADMIN	Vai trò người dùng — RBAC 3 cấp theo FR-AUTH-02 ▶ Bản kế hoạch Topic_3 chỉ có 2 vai trò (customer/admin). Bản đặc tả v2 yêu cầu 3 vai trò.	FR-AUT-H-02, NFR-02.2
<code>order_status</code>	PENDING_PAYMENT • CONFIRMED • SHIPPING • DELIVERED • CANCELLED	Vòng đời đơn hàng — state machine một chiều ▶ Thay cho bộ pending/paid/reserved/fulfilled/cancelled trong Topic_3 (xung đột, đã chọn theo đặc tả v2).	FR-ORD-01, FR-ORD-03, FR-ORD-05, FR-PAY-02
<code>reservation_status</code>	HOLDING • COMMITTED • RELEASED • EXPIRED	Trạng thái bản ghi giữ chỗ tồn kho ▶ Topic_3 thiếu giá trị EXPIRED — bắt buộc phải có để phân biệt 'hết hạn tự động' với 'bị hủy chủ động'.	FR-ORD-02, FR-ORD-04, FR-PAY-02
<code>payment_status</code>	PENDING • SUCCESS • FAILED	Trạng thái giao dịch thanh toán mô phỏng	FR-PAY-01, FR-PAY-02
<code>payment_method</code>	MOCK_CARD • MOCK_EWALLET • COD	Phương thức thanh toán (đều là mô phỏng) ▶ Tiền tố MOCK_ để nhấn mạnh CON-05: không tích hợp cổng thật.	FR-PAY-01, CON-05
<code>alert_level</code>	WARNING • CRITICAL	Mức độ cảnh báo tồn kho thấp ▶ WARNING khi available <= threshold; CRITICAL khi available = 0.	FR-NOT-01
<code>alert_status</code>	OPEN • RESOLVED	Trạng thái xử lý cảnh báo	FR-NOT-01
<code>inventory_change_reason</code>	RESERVE • COMMIT • RELEASE • EXPIRE • RESTOCK • MANUAL_ADJUST	Lý do thay đổi tồn kho — dùng cho audit trail ▶ Cho phép truy vết chính xác mọi biến động kho, phục vụ NFR-03.2 và NFR-08.2.	FR-INV-01, NFR-03.2
<code>chat_role</code>	USER • ASSISTANT • SYSTEM	Vai trò của một lượt tin nhắn trong hội thoại chatbot	FR-BOT-02
<code>cart_status</code>	ACTIVE • CONVERTED • ABANDONED	Trạng thái giỏ hàng server-side ▶ Chỉ dùng nếu nhóm chốt lưu giỏ	FR-CART-01

Tên ENUM	Các giá trị	Mô tả & ghi chú	Truy vết
		hàng server-side (mục 9 đặc tả còn để ngỏ).	

5. ĐẶC TẢ CHI TIẾT 18 BẢNG

Mỗi bảng được trình bày gồm: mục đích, module sở hữu, danh sách cột đầy đủ và các yêu cầu được truy vết. Ký hiệu trong cột “Khóa”: PK = khóa chính, FK = khóa ngoại, UQ = ràng buộc duy nhất, GEN = cột sinh tự động. Cột “Null” đánh dấu ✓ nghĩa là cho phép để trống.

5.1. Nhóm A — Identity & Access (sở hữu: P4)

Bảng 1. users — Người dùng hệ thống

Mục đích: Lưu tài khoản của cả 3 vai trò. Mật khẩu chỉ lưu dạng băm bcrypt/argon2, không bao giờ lưu plaintext.

Truy vết yêu cầu: *FR-AUTH-01, FR-AUTH-02, NFR-02.1* **Module sở hữu:** **P4**

Khóa	Tên cột	Kiểu dữ liệu	Null	Mặc định	Ý nghĩa
PK	id	UUID		gen_random_uuid()	Khóa chính
UQ	email	VARCHAR(255)		–	Email đăng nhập, duy nhất toàn hệ thống
–	password_hash	VARCHAR(255)		–	Mật khẩu đã băm (bcrypt cost>=10 hoặc argon2id)
–	full_name	VARCHAR(150)		–	Họ tên hiển thị
–	phone	VARCHAR(20)	✓	–	Số điện thoại liên hệ
–	role	user_role		'CUSTOMER'	Vai trò RBAC; STAFF/ADMIN chỉ tạo bằng seed hoặc do Admin tạo
–	is_active	BOOLEAN		TRUE	Cho phép vô hiệu hóa tài khoản mà không xóa dữ liệu
–	created_at	TIMESTAMPTZ		NOW()	Thời điểm tạo
–	updated_at	TIMESTAMPTZ		NOW()	Thời điểm cập nhật gần nhất

Bảng 2. refresh_tokens — Refresh token

Mục đích: Hỗ trợ cơ chế JWT + Refresh Token đã chốt trong kế hoạch. Lưu dạng băm để nếu lộ DB cũng không dùng lại được token.

Truy vết yêu cầu: *FR-AUTH-01* **Module sở hữu:** **P4**

Khóa	Tên cột	Kiểu dữ liệu	Null	Mặc định	Ý nghĩa
PK	id	UUID		gen_random_uuid()	Khóa chính
FK	user_id	UUID		–	→ users.id, ON DELETE CASCADE
UQ	token_hash	VARCHAR(255)		–	SHA-256 của refresh token, không lưu token gốc
–	expires_at	TIMESTAMPTZ		–	Hạn dùng của refresh token
–	revoked_at	TIMESTAMPTZ	✓	–	Thời điểm thu hồi (logout / đổi mật khẩu)
–	created_at	TIMESTAMPTZ		NOW()	Thời điểm phát hành

5.2. Nhóm B — Catalog (sở hữu: P4)

Bảng 3. categories — Danh mục sản phẩm

Mục đích: Cây danh mục phân cấp bằng self-reference parent_id. Cho phép danh mục cha - con nhiều cấp.

Truy vết yêu cầu: *FR-CAT-01, FR-ADM-01* **Module sở hữu: P4**

Khóa	Tên cột	Kiểu dữ liệu	Null	Mặc định	Ý nghĩa
PK	id	UUID		gen_random_uuid()	Khóa chính
—	name	VARCHAR(150)		—	Tên danh mục
UQ	slug	VARCHAR(160)		—	Định danh thân thiện URL
FK	parent_id	UUID	✓	—	→ categories.id (tự tham chiếu); NULL = danh mục gốc
—	is_active	BOOLEAN		TRUE	Ẩn/hiện danh mục
—	created_at	TIMESTAMPTZ		NOW()	Thời điểm tạo
—	updated_at	TIMESTAMPTZ		NOW()	Thời điểm cập nhật

Bảng 4. products — Sản phẩm (catalog gốc)

Mục đích: KHÔNG chứa số lượng tồn kho — số lượng nằm hoàn toàn ở bảng inventory. Đây là nguyên tắc tách biệt cốt lõi của mô hình đa kho.

Truy vết yêu cầu: *FR-CAT-01, FR-CAT-02, FR-CAT-03, FR-ADM-01* **Module sở hữu: P4**

Khóa	Tên cột	Kiểu dữ liệu	Null	Mặc định	Ý nghĩa
PK	id	UUID		gen_random_uuid()	Khóa chính
UQ	sku	VARCHAR(64)		—	Mã sản phẩm nội bộ, duy nhất
—	name	VARCHAR(255)		—	Tên sản phẩm
UQ	slug	VARCHAR(280)		—	Định danh thân thiện URL
—	description	TEXT	✓	—	Mô tả chi tiết
—	base_price	NUMERIC(14,2)		—	Đơn giá gốc — BẮT BUỘC dùng NUMERIC, không dùng FLOAT
—	discount_percent	NUMERIC(5,2)		0	Phần trăm giảm giá hiện hành (0–100)
FK	category_id	UUID	✓	—	→ categories.id, ON DELETE SET NULL
—	image_url	VARCHAR(500)	✓	—	Ảnh đại diện sản phẩm
—	is_active	BOOLEAN		TRUE	Ngừng kinh doanh thì set FALSE thay vì xóa
—	created_at	TIMESTAMPTZ		NOW()	Thời điểm tạo
—	updated_at	TIMESTAMPTZ		NOW()	Thời điểm cập nhật

5.3. Nhóm C — Warehouse & Inventory (sở hữu: P4) — LỖI KỸ THUẬT

Bảng 5. warehouses — Kho hàng khu vực

Mục đích: Danh sách các kho vật lý theo khu vực. Không được xóa kho đang được inventory/reservation tham chiếu (ON DELETE RESTRICT).

Truy vết yêu cầu: *FR-ADM-02, FR-INV-01, URq-ADM-04* **Module sở hữu:** **P4**

Khóa	Tên cột	Kiểu dữ liệu	Null	Mặc định	Ý nghĩa
PK	id	UUID		<code>gen_random_uuid()</code>	Khóa chính
UQ	code	VARCHAR(32)		—	Mã kho ngắn, ví dụ WH-HCM, WH-HN, WH-DN
—	name	VARCHAR(150)		—	Tên kho
—	region	VARCHAR(100)		—	Khu vực địa lý — dùng cho thuật toán chọn kho gần nhất
—	address	VARCHAR(500)	✓	—	Địa chỉ kho
—	capacity	INTEGER	✓	—	Sức chứa tối đa (tham khảo, không ràng buộc cứng)
—	default_low_stock_threshold	INTEGER		10	Ngưỡng cảnh báo mặc định cấp kho
—	is_active	BOOLEAN		TRUE	Tạm ngừng hoạt động kho
—	created_at	TIMESTAMPTZ		<code>NOW()</code>	Thời điểm tạo
—	updated_at	TIMESTAMPTZ		<code>NOW()</code>	Thời điểm cập nhật

Bảng 6. inventory — Tồn kho (BẢNG TRUNG TÂM)

Mục đích: Bảng trung gian hiện thực quan hệ N–N giữa products và warehouses. Đây là bảng bị SELECT ... FOR UPDATE khóa dòng trong Stock Reservation Engine — trái tim kỹ thuật của toàn đồ án.

Truy vết yêu cầu: *FR-INV-01, FR-INV-02, FR-ORD-02, FR-ORD-04, NFR-04.1* **Module sở hữu:** **P4**

Khóa	Tên cột	Kiểu dữ liệu	Null	Mặc định	Ý nghĩa
PK	id	BIGSERIAL		—	Khóa chính KIỂU SỐ TĂNG DẦN — cố ý dùng BIGSERIAL thay UUID để sắp xếp thứ tự khóa chống deadlock
FK	product_id	UUID		—	→ products.id, ON DELETE CASCADE
FK	warehouse_id	UUID		—	→ warehouses.id, ON DELETE RESTRICT
—	stock	INTEGER		0	Tồn kho vật lý thực tế; CHECK stock >= 0
—	reserved_stock	INTEGER		0	Số lượng đang bị giữ chỗ; CHECK reserved_stock >= 0 AND reserved_stock <= stock
GEN	available_stock	INTEGER		GENERATED	Cột sinh tự động = stock – reserved_stock (GENERATED ALWAYS STORED)
—	threshold	INTEGER	✓	—	Ngưỡng cảnh báo riêng cho cấp SP–kho; NULL = dùng mặc định của kho
—	version	INTEGER		0	Optimistic locking dự phòng (defense-in-depth)
—	last_restocked_at	TIMESTAMPTZ	✓	—	Lần nhập hàng gần nhất
—	created_at	TIMESTAMPTZ		<code>NOW()</code>	Thời điểm tạo

Khóa	Tên cột	Kiểu dữ liệu	Null	Mặc định	Ý nghĩa
—	updated_at	TIMESTAMPZ		NOW()	Thời điểm cập nhật

Bảng 7. inventory_audit_log — Nhật ký biến động tồn kho

Mục đích: Ghi lại MỌI thay đổi tồn kho kèm giá trị trước/sau. Là bằng chứng đối soát cho NFR-03.2 (ACID) và NFR-08.2 (toàn vẹn dưới tải cao).

Truy vết yêu cầu: *FR-INV-01, NFR-03.2, NFR-08.2* **Module sở hữu:** **P4**

Khóa	Tên cột	Kiểu dữ liệu	Null	Mặc định	Ý nghĩa
PK	id	BIGSERIAL		—	Khóa chính
FK	inventory_id	BIGINT		—	→ inventory.id, ON DELETE CASCADE
—	reason	inventory_change_reason		—	Lý do biến động
—	stock_before	INTEGER		—	Tồn vật lý trước thay đổi
—	stock_after	INTEGER		—	Tồn vật lý sau thay đổi
—	reserved_before	INTEGER		—	Số giữ chỗ trước thay đổi
—	reserved_after	INTEGER		—	Số giữ chỗ sau thay đổi
—	quantity_delta	INTEGER		—	Lượng thay đổi (dương/âm)
—	reference_type	VARCHAR(40)	✓	—	Loại đối tượng gây ra thay đổi: ORDER / RESERVATION / MANUAL
—	reference_id	VARCHAR(64)	✓	—	ID của đối tượng đó
FK	actor_id	UUID	✓	—	→ users.id; NULL nếu do hệ thống/cron job
—	note	VARCHAR(500)	✓	—	Ghi chú thêm
—	created_at	TIMESTAMPZ		NOW()	Thời điểm ghi log

Vì sao inventory.id dùng BIGSERIAL chứ không phải UUID

Đây là quyết định thiết kế có chủ đích, không phải thiếu nhất quán. Đặc tả FR-ORD-02 yêu cầu “sắp xếp inventory_id tăng dần rồi SELECT ... FOR UPDATE” để tránh deadlock khi một đơn hàng có nhiều sản phẩm. UUID v4 là ngẫu nhiên nên thứ tự sắp xếp của nó không mang ý nghĩa ổn định giữa các lần chạy. BIGSERIAL cho một thứ tự tổng thể tự nhiên, đơn giản và luôn nhất quán — mọi transaction khóa theo cùng một trật tự thì vòng chờ chéo không thể hình thành.

5.4. Nhóm D — Cart (sở hữu: P1) — tùy chọn

Bảng 8. carts — Giỏ hàng

Mục đích: TÙY CHỌN — chỉ tạo nếu nhóm chốt lưu giỏ hàng server-side. Giỏ hàng TUYỆT ĐỐI KHÔNG giữ chỗ tồn kho (chỉ FR-ORD-02 mới giữ chỗ).

Truy vết yêu cầu: *FR-CART-01* **Module sở hữu:** **P1**

Khóa	Tên cột	Kiểu dữ liệu	Null	Mặc định	Ý nghĩa
PK	id	UUID		gen_random_uuid()	Khóa chính
FK	user_id	UUID		—	→ users.id, ON DELETE CASCADE
—	status	cart_status		'ACTIVE'	Mỗi user chỉ có tối đa 1 giỏ ACTIVE (partial unique index)
—	created_at	TIMESTAMPTZ		NOW()	Thời điểm tạo
—	updated_at	TIMESTAMPTZ		NOW()	Thời điểm cập nhật

Bảng 9. cart_items — Dòng sản phẩm trong giỏ

Mục đích: Chỉ lưu productId và quantity. KHÔNG lưu giá — giá luôn được server tính lại tại thời điểm tạo đơn (FR-CART-03).

Truy vết yêu cầu: *FR-CART-01, FR-CART-03* **Module sở hữu:** **P1**

Khóa	Tên cột	Kiểu dữ liệu	Null	Mặc định	Ý nghĩa
PK	id	UUID		gen_random_uuid()	Khóa chính
FK	cart_id	UUID		—	→ carts.id, ON DELETE CASCADE
FK	product_id	UUID		—	→ products.id, ON DELETE CASCADE
—	quantity	INTEGER		1	CHECK quantity > 0
—	added_at	TIMESTAMPTZ		NOW()	Thời điểm thêm vào giỏ
—	updated_at	TIMESTAMPTZ		NOW()	Thời điểm sửa số lượng

5.5. Nhóm E — Order & Reservation (sở hữu: P3) — LỖI KỸ THUẬT

Bảng 10. orders — Đơn hàng

Mục đích: Lưu snapshot toàn bộ thông tin giao hàng và tổng tiền tại thời điểm đặt. Tổng tiền KHÔNG được tính lại từ giá sản phẩm hiện tại (vì giá có thể đã đổi).

Truy vết yêu cầu: *FR-ORD-01, FR-ORD-03, FR-ORD-05, FR-PAY-02, FR-DASH-02* **Module sở hữu: P3**

Khóa	Tên cột	Kiểu dữ liệu	Null	Mặc định	Ý nghĩa
PK	id	UUID		gen_random_uuid()	Khóa chính
UQ	order_code	VARCHAR(32)		—	Mã đơn hiển thị cho khách, ví dụ ORD-2026-000088
FK	user_id	UUID		—	→ users.id, ON DELETE RESTRICT
—	status	order_status		'PENDING_PAYMENT'	Trạng thái theo state machine một chiều
—	subtotal	NUMERIC(14,2)		0	Tổng tiền hàng trước giảm giá
—	discount_total	NUMERIC(14,2)		0	Tổng giảm giá
—	shipping_fee	NUMERIC(14,2)		0	Phí vận chuyển (quy tắc do nhóm chốt)
—	grand_total	NUMERIC(14,2)		0	Tổng phải trả = subtotal – discount_total + shipping_fee
—	currency	CHAR(3)		'VND'	Mã tiền tệ ISO 4217
—	shipping_recipient	VARCHAR(150)		—	Tên người nhận (snapshot)
—	shipping_phone	VARCHAR(20)		—	SĐT người nhận (snapshot)
—	shipping_address	VARCHAR(500)		—	Địa chỉ giao hàng (snapshot)
—	shipping_region	VARCHAR(100)	✓	—	Khu vực giao — đầu vào cho thuật toán chọn kho
—	reservation_expires_at	TIMESTAMPZ	✓	—	Hạn giữ chỗ chung của đơn (= min expires_at các reservation)
—	cancelled_reason	VARCHAR(255)	✓	—	Lý do hủy: EXPIRED / PAYMENT_FAILED / USER_CANCELLED
—	confirmed_at	TIMESTAMPZ	✓	—	Thời điểm thanh toán thành công
—	cancelled_at	TIMESTAMPZ	✓	—	Thời điểm hủy
—	created_at	TIMESTAMPZ		NOW()	Thời điểm tạo đơn
—	updated_at	TIMESTAMPZ		NOW()	Thời điểm cập nhật

Bảng 11. order_items — Dòng sản phẩm trong đơn

Mục đích: Mỗi dòng gắn với MỘT kho cụ thể — đây là điểm cho phép 1 đơn hàng lấy hàng từ nhiều kho khác nhau. Mọi thông tin sản phẩm đều là snapshot.

Truy vết yêu cầu: *FR-ORD-01, FR-ADM-01, FR-DASH-02* **Module sở hữu: P3**

Khóa	Tên cột	Kiểu dữ liệu	Null	Mặc định	Ý nghĩa
PK	id	UUID		gen_random_uuid()	Khóa chính
FK	order_id	UUID		—	→ orders.id, ON DELETE CASCADE
FK	product_id	UUID		—	→ products.id, ON DELETE RESTRICT

Khóa	Tên cột	Kiểu dữ liệu	Null	Mặc định	Ý nghĩa
FK	warehouse_id	UUID		–	→ warehouses.id, ON DELETE RESTRICT — kho được phân bổ
FK	inventory_id	BIGINT		–	→ inventory.id — trở thẳng dòng tồn kho bị khóa
–	product_name	VARCHAR(255)		–	SNAPSHOT tên sản phẩm lúc đặt
–	sku	VARCHAR(64)		–	SNAPSHOT mã SKU lúc đặt
–	unit_price	NUMERIC(14,2)		–	SNAPSHOT đơn giá lúc đặt — không đổi dù giá sản phẩm sau này thay đổi
–	discount_amount	NUMERIC(14,2)		0	SNAPSHOT giảm giá áp dụng cho dòng này
–	quantity	INTEGER		–	CHECK quantity > 0
–	line_total	NUMERIC(14,2)		–	= unit_price × quantity – discount_amount
–	created_at	TIMESTAMPZ		NOW()	Thời điểm tạo

Bảng 12. stock_reservations — Giữ chỗ tồn kho (CHỐNG OVERSELL)

Mục đích: Bảng quan trọng thứ hai của đồ án. Khi khách đặt hàng, hệ thống KHÔNG trừ ngay vào stock mà tạo bản ghi giữ chỗ có TTL 15 phút và tăng reserved_stock. Chỉ khi thanh toán thành công mới trừ stock thật.

Truy vết yêu cầu: *FR-ORD-02, FR-ORD-04, FR-PAY-02, NFR-04.1, NFR-04.3* **Module sở hữu: P3**

Khóa	Tên cột	Kiểu dữ liệu	Null	Mặc định	Ý nghĩa
PK	id	BIGSERIAL		–	Khóa chính
FK	order_id	UUID		–	→ orders.id, ON DELETE CASCADE
FK	order_item_id	UUID	✓	–	→ order_items.id — truy vết tới đúng dòng hàng
FK	inventory_id	BIGINT		–	→ inventory.id, ON DELETE RESTRICT
–	quantity	INTEGER		–	Số lượng giữ chỗ; CHECK quantity > 0
–	status	reservation_status		'HOLDING'	HOLDING → COMMITTED / RELEASED / EXPIRED
–	expires_at	TIMESTAMPZ		–	Hạn giữ chỗ = created_at + TTL (mặc định 15 phút)
–	committed_at	TIMESTAMPZ	✓	–	Thời điểm chốt đơn, trừ stock thật
–	released_at	TIMESTAMPZ	✓	–	Thời điểm hoàn trả reserved_stock
–	created_at	TIMESTAMPZ		NOW()	Thời điểm tạo giữ chỗ
–	updated_at	TIMESTAMPZ		NOW()	Thời điểm cập nhật

Bảng 13. order_status_history — Lịch sử chuyển trạng thái đơn

Mục đích: Ghi lại mọi lần đơn hàng đổi trạng thái, kèm người thực hiện. Đây là nguồn dữ liệu để khách theo dõi đơn và Chatbot trả lời 'đơn tới đâu rồi'.

Truy vết yêu cầu: *FR-ORD-05, FR-ORD-03, FR-BOT-03* **Module sở hữu: P3**

Khóa	Tên cột	Kiểu dữ liệu	Null	Mặc định	Ý nghĩa
PK	id	BIGSERIAL		–	Khóa chính

Khóa	Tên cột	Kiểu dữ liệu	Null	Mặc định	Ý nghĩa
FK	order_id	UUID		–	→ orders.id, ON DELETE CASCADE
–	from_status	order_status	✓	–	Trạng thái cũ; NULL ở bản ghi đầu tiên
–	to_status	order_status		–	Trạng thái mới
FK	actor_id	UUID	✓	–	→ users.id; NULL nếu do hệ thống (cron/payment callback)
–	reason	VARCHAR(255)	✓	–	Lý do / ghi chú khi chuyển trạng thái
–	created_at	TIMESTAMPZ		NOW()	Thời điểm chuyển

Vì sao `stock_reservations` tách riêng khỏi `order_items`

Có thể nghĩ rằng chỉ cần thêm cột `reserved` vào `order_items` là đủ. Nhưng tách bảng riêng mang lại ba lợi ích thiết thực:

1. Vòng đời khác nhau — một `order_item` tồn tại vĩnh viễn, còn `reservation` chỉ sống 15 phút rồi chuyển trạng thái. Trộn chung sẽ làm bảng `order_items` bị cập nhật liên tục bởi cron job.
2. Cron job FR-ORD-04 quét được bằng một index rất hẹp: `idx_res_cleanup` chỉ chứa các dòng đang HOLDING, thay vì phải quét toàn bộ `order_items`.
3. Cho phép một `order_item` có nhiều `reservation` nếu sau này nhóm mở rộng sang tách đơn theo nhiều kho.

5.6. Nhóm F — Payment (sở hữu: P3)

Bảng 14. payments — Giao dịch thanh toán (mô phỏng)

Mục đích: transaction_ref có ràng buộc UNIQUE — chính ràng buộc này là cơ chế bảo đảm idempotency ở tầng CSDL: callback lặp lại sẽ bị chặn bởi database chứ không chỉ bởi code.

Truy vết yêu cầu: *FR-PAY-01, FR-PAY-02, NFR-04.2, NFR-02.3, NFR-03.1* **Module sở hữu:** **P3**

Khóa	Tên cột	Kiểu dữ liệu	Null	Mặc định	Ý nghĩa
PK	id	UUID		gen_random_uuid()	Khóa chính
FK	order_id	UUID		—	→ orders.id, ON DELETE RESTRICT
UQ	transaction_ref	VARCHAR(64)		—	KHÓA IDEMPOTENCY — UNIQUE, callback lặp bị DB chặn
—	amount	NUMERIC(14,2)		—	Số tiền; phải khớp orders.grand_total
—	currency	CHAR(3)		'VND'	Mã tiền tệ
—	method	payment_method		—	Phương thức thanh toán mô phỏng
—	status	payment_status		'PENDING'	PENDING → SUCCESS / FAILED
—	failure_reason	VARCHAR(255)	✓	—	Lý do thất bại (mô phỏng)
—	gateway_response	JSONB	✓	—	Payload thô từ mock gateway — TUYỆT ĐỐI không chứa số thẻ/CVV
—	callback_count	INTEGER		0	Đếm số lần callback nhận được — bằng chứng cho test replay NFR-04.2
—	processed_at	TIMESTAMPTZ	✓	—	Thời điểm xử lý callback thành công lần đầu
—	created_at	TIMESTAMPTZ		NOW()	Thời điểm khởi tạo giao dịch
—	updated_at	TIMESTAMPTZ		NOW()	Thời điểm cập nhật

5.7. Nhóm G — Notification (sở hữu: P5)

Bảng 15. stock_alerts — Cảnh báo tồn kho thấp

Mục đích: Ghi lại cảnh báo đã phát. Nhờ bảng này mà khi WebSocket mất kết nối, Admin tải lại trang vẫn thấy đầy đủ cảnh báo (yêu cầu ở luồng thay thế FR-NOT-01).

Truy vết yêu cầu: *FR-NOT-01, FR-DASH-01, IF-04* **Module sở hữu:** **P5**

Khóa	Tên cột	Kiểu dữ liệu	Null	Mặc định	Ý nghĩa
PK	id	BIGSERIAL		—	Khóa chính
FK	inventory_id	BIGINT		—	→ inventory.id, ON DELETE CASCADE
—	alert_level	alert_level		'WARNING'	WARNING / CRITICAL
—	status	alert_status		'OPEN'	OPEN / RESOLVED
—	stock_at_trigger	INTEGER		—	Tồn vật lý tại thời điểm phát cảnh báo
—	available_at_trigger	INTEGER		—	Tồn khả dụng tại thời điểm phát cảnh báo
—	threshold_at_trigger	INTEGER		—	Ngưỡng áp dụng tại thời điểm đó
—	triggered_at	TIMESTAMPZ		NOW()	Thời điểm phát cảnh báo
—	resolved_at	TIMESTAMPZ	✓	—	Thời điểm đánh dấu đã xử lý
FK	resolved_by	UUID	✓	—	→ users.id — ai đã xử lý

5.8. Nhóm H — AI Chatbot RAG (sở hữu: P5)

Bảng 16. kb_documents — Tài liệu tri thức (nguồn RAG tĩnh)

Mục đích: Lưu FAQ/chính sách đã được chia đoạn (chunk) và vector hóa. Cột embedding dùng kiểu vector của extension pgvector.

Truy vết yêu cầu: *FR-BOT-01, FR-BOT-02* **Module sở hữu:** **P5**

Khóa	Tên cột	Kiểu dữ liệu	Null	Mặc định	Ý nghĩa
PK	id	UUID		gen_random_uuid()	Khóa chính
—	title	VARCHAR(255)		—	Tiêu đề tài liệu
—	doc_type	VARCHAR(50)		—	FAQ / RETURN_POLICY / SHIPPING_POLICY / GENERAL
—	chunk_index	INTEGER		0	Thứ tự đoạn trong tài liệu gốc
—	content	TEXT		—	Nội dung đoạn văn bản
—	embedding	vector(1024)	✓	—	Vector nhúng — số chiều tùy mô hình embedding đã chọn
—	token_count	INTEGER	✓	—	Số token của đoạn, dùng kiểm soát chi phí (CON-07)
—	source_url	VARCHAR(500)	✓	—	Nguồn tài liệu gốc
—	is_active	BOOLEAN		TRUE	Cho phép tắt 1 đoạn mà không xóa
—	created_at	TIMESTAMPTZ		NOW()	Thời điểm tạo
—	updated_at	TIMESTAMPTZ		NOW()	Thời điểm cập nhật

Bảng 17. chat_sessions — Phiên hội thoại Chatbot

Mục đích: user_id cho phép NULL vì FR-BOT-05 quy định khách CHƯA đăng nhập vẫn hỏi được chính sách và tồn kho công khai.

Truy vết yêu cầu: *FR-BOT-05, FR-BOT-04, NFR-02.1* **Module sở hữu:** **P5**

Khóa	Tên cột	Kiểu dữ liệu	Null	Mặc định	Ý nghĩa
PK	id	UUID		gen_random_uuid()	Khóa chính
FK	user_id	UUID	✓	—	→ users.id; NULL = khách vắng lai chưa đăng nhập
UQ	session_token	VARCHAR(100)		—	Định danh phiên phía client
—	escalated_to_human	BOOLEAN		FALSE	Đã đề xuất chuyển CSKH hay chưa (FR-BOT-04)
—	message_count	INTEGER		0	Số lượt tin nhắn — dùng cho rate limit CON-07
—	started_at	TIMESTAMPTZ		NOW()	Thời điểm bắt đầu phiên
—	ended_at	TIMESTAMPTZ	✓	—	Thời điểm kết thúc phiên

Bảng 18. chat_messages — Tin nhắn hội thoại

Mục đích: Cột retrieved_context và tool_calls lưu lại đúng ngữ cảnh đã đưa vào prompt — đây là bằng chứng để chấm điểm NFR-04.4 (chatbot không bịa dữ liệu) và debug khi trả lời sai.

Truy vết yêu cầu: *FR-BOT-01, FR-BOT-02, FR-BOT-03, FR-BOT-04, NFR-04.4, NFR-07.2* Module sở hữu: **P5**

Khóa	Tên cột	Kiểu dữ liệu	Null	Mặc định	Ý nghĩa
PK	id	BIGSERIAL		–	Khóa chính
FK	session_id	UUID		–	→ chat_sessions.id, ON DELETE CASCADE
–	role	chat_role		–	USER / ASSISTANT / SYSTEM
–	content	TEXT		–	Nội dung tin nhắn
–	retrieved_context	JSONB	✓	–	Ngữ cảnh đã truy xuất (id tài liệu, điểm tương đồng, dữ liệu đơn/kho)
–	tool_calls	JSONB	✓	–	Các tool đã gọi: lookupOrder / checkInventory + tham số
–	is_fallback	BOOLEAN		FALSE	Lượt trả lời này có rơi vào fallback không
–	latency_ms	INTEGER	✓	–	Độ trễ phản hồi — đo cho NFR-01.3
–	token_usage	INTEGER	✓	–	Token đã dùng — kiểm soát chi phí CON-07
–	created_at	TIMESTAMPZ		NOW()	Thời điểm tạo

6. QUAN HỆ GIỮA CÁC BẢNG

Bảng dưới liệt kê toàn bộ 25 khóa ngoại. Cột “Khi xóa bản ghi cha” đặc biệt quan trọng: RESTRICT được dùng cho mọi dữ liệu giao dịch để không bao giờ mất lịch sử bán hàng, CASCADE chỉ dùng cho dữ liệu con phụ thuộc hoàn toàn vào cha.

Bảng con (chứa FK)	Cột khóa ngoại	Trở tới bảng cha	Bản số	Khi xóa cha	Ý nghĩa nghiệp vụ
refresh_tokens	user_id	users.id	N-1	CASCADE	Một user có nhiều refresh token
categories	parent_id	categories.id	N-1	SET NULL	Tự tham chiếu — cây danh mục cha/con
products	category_id	categories.id	N-1	SET NULL	Một danh mục chứa nhiều sản phẩm
inventory	product_id	products.id	N-1	CASCADE	Một sản phẩm có mặt ở nhiều kho
inventory	warehouse_id	warehouses.id	N-1	RESTRICT	Một kho chứa nhiều sản phẩm
inventory_audit_log	inventory_id	inventory.id	N-1	CASCADE	Một dòng tồn kho có nhiều bản ghi lịch sử
inventory_audit_log	actor_id	users.id	N-1	SET NULL	Người thực hiện thay đổi
carts	user_id	users.id	N-1	CASCADE	Một user có nhiều giỏ (chỉ 1 giỏ ACTIVE)
cart_items	cart_id	carts.id	N-1	CASCADE	Một giỏ có nhiều dòng sản phẩm
cart_items	product_id	products.id	N-1	CASCADE	Sản phẩm được tham chiếu trong giỏ
orders	user_id	users.id	N-1	RESTRICT	Một khách có nhiều đơn hàng
order_items	order_id	orders.id	N-1	CASCADE	Một đơn có nhiều dòng hàng
order_items	product_id	products.id	N-1	RESTRICT	Không xóa được sản phẩm đã từng bán
order_items	warehouse_id	warehouses.id	N-1	RESTRICT	Dòng hàng được phân bổ về 1 kho cụ thể
order_items	inventory_id	inventory.id	N-1	RESTRICT	Trở thẳng dòng tồn kho bị trừ
stock_reservations	order_id	orders.id	N-1	CASCADE	Một đơn có nhiều bản ghi giữ chỗ
stock_reservations	order_item_id	order_items.id	N-1	CASCADE	Giữ chỗ tương ứng từng dòng hàng
stock_reservations	inventory_id	inventory.id	N-1	RESTRICT	Giữ chỗ trên 1 dòng tồn kho
order_status_history	order_id	orders.id	N-1	CASCADE	Một đơn có nhiều lần đổi trạng thái
order_status_history	actor_id	users.id	N-1	SET NULL	Người thực hiện chuyển trạng thái
payments	order_id	orders.id	N-1	RESTRICT	Một đơn có thể có nhiều lần thanh toán (retry)
stock_alerts	inventory_id	inventory.id	N-1	CASCADE	Một dòng tồn kho có nhiều cảnh báo
stock_alerts	resolved_by	users.id	N-1	SET NULL	Người đánh dấu đã xử lý
chat_sessions	user_id	users.id	N-1	SET NULL	Một user có nhiều phiên chat; NULL = khách vắng lại

Bảng con (chứa FK)	Cột khóa ngoại	Trỏ tới bảng cha	Bản số	Khi xóa cha	Ý nghĩa nghiệp vụ
chat_messages	session_id	chat_sessions.id	N-1	CASCADE	Một phiên có nhiều tin nhắn

6.1. Quan hệ N–N cốt lõi

Quan hệ quan trọng nhất của toàn hệ thống là N–N giữa products và warehouses, được hiện thực qua bảng trung gian inventory:

- Một sản phẩm có thể nằm ở nhiều kho khác nhau, mỗi kho một mức tồn riêng.
- Một kho chứa nhiều sản phẩm khác nhau.
- Ràng buộc UNIQUE (product_id, warehouse_id) bảo đảm mỗi cặp chỉ tồn tại đúng một dòng — nếu thiếu ràng buộc này, cùng một sản phẩm ở cùng một kho có thể bị tách thành nhiều dòng và mọi phép cộng tồn kho sẽ sai.

6.2. Quan hệ tự tham chiếu

categories.parent_id trỏ về chính categories.id, tạo cây danh mục nhiều cấp. Ràng buộc chk_categories_no_self_parent chặn trường hợp một danh mục tự nhận mình làm cha. Lưu ý: ràng buộc này không chặn được vòng lặp dài hơn ($A \rightarrow B \rightarrow A$); nếu nhóm cần chặn triệt để thì phải kiểm tra ở tầng ứng dụng bằng truy vấn đệ quy.

7. RÀNG BUỘC TOÀN VỆN & CHIẾN LƯỢC INDEX

7.1. Các ràng buộc chống oversell ở tầng CSDL

Đây là lớp phòng thủ cuối cùng. Kể cả khi code ứng dụng có lỗi logic, các ràng buộc sau vẫn khiến giao dịch sai bị từ chối:

Ràng buộc	Biểu thức	Chặn được tình huống gì
<code>chk_inv_stock_nonneg</code>	<code>stock >= 0</code>	Trừ kho quá tay làm tồn kho âm — dấu hiệu kinh điển của oversell.
<code>chk_inv_reserved_nonneg</code>	<code>reserved_stock >= 0</code>	Giải phóng giữ chỗ hai lần (cron job chạy trùng) làm <code>reserved_stock</code> âm.
<code>chk_inv_reserved_le_stock</code>	<code>reserved_stock <= stock</code>	Giữ chỗ nhiều hơn số hàng thực có — chính là oversell ở dạng giữ chỗ.
<code>uq_inventory_product_warehouse</code>	<code>UNIQUE (product_id, warehouse_id)</code>	Cùng sản phẩm cùng kho bị tách thành hai dòng, khiến mọi phép cộng tồn kho sai.
<code>chk_orders_grand_total</code>	<code>grand_total = subtotal - discount_total + shipping_fee</code>	Tổng tiền không khớp các thành phần — chặn lỗi tính giá của Pricing Engine.
<code>chk_oi_line_total</code>	<code>line_total = unit_price × quantity - discount_amount</code>	Thành tiền từng dòng bị tính sai.
<code>uq_payments_one_success_per_order</code>	<code>UNIQUE (order_id) WHERE status = 'SUCCESS'</code>	Một đơn có hai giao dịch thành công — tức trừ kho hai lần do callback lặp.
<code>uq_alerts_one_open_per_inventory</code>	<code>UNIQUE (inventory_id) WHERE status = 'OPEN'</code>	Spam cảnh báo: mỗi lần trừ kho lại tạo thêm một cảnh báo mới cho cùng một mặt hàng.

7.2. Các index quan trọng và lý do tồn tại

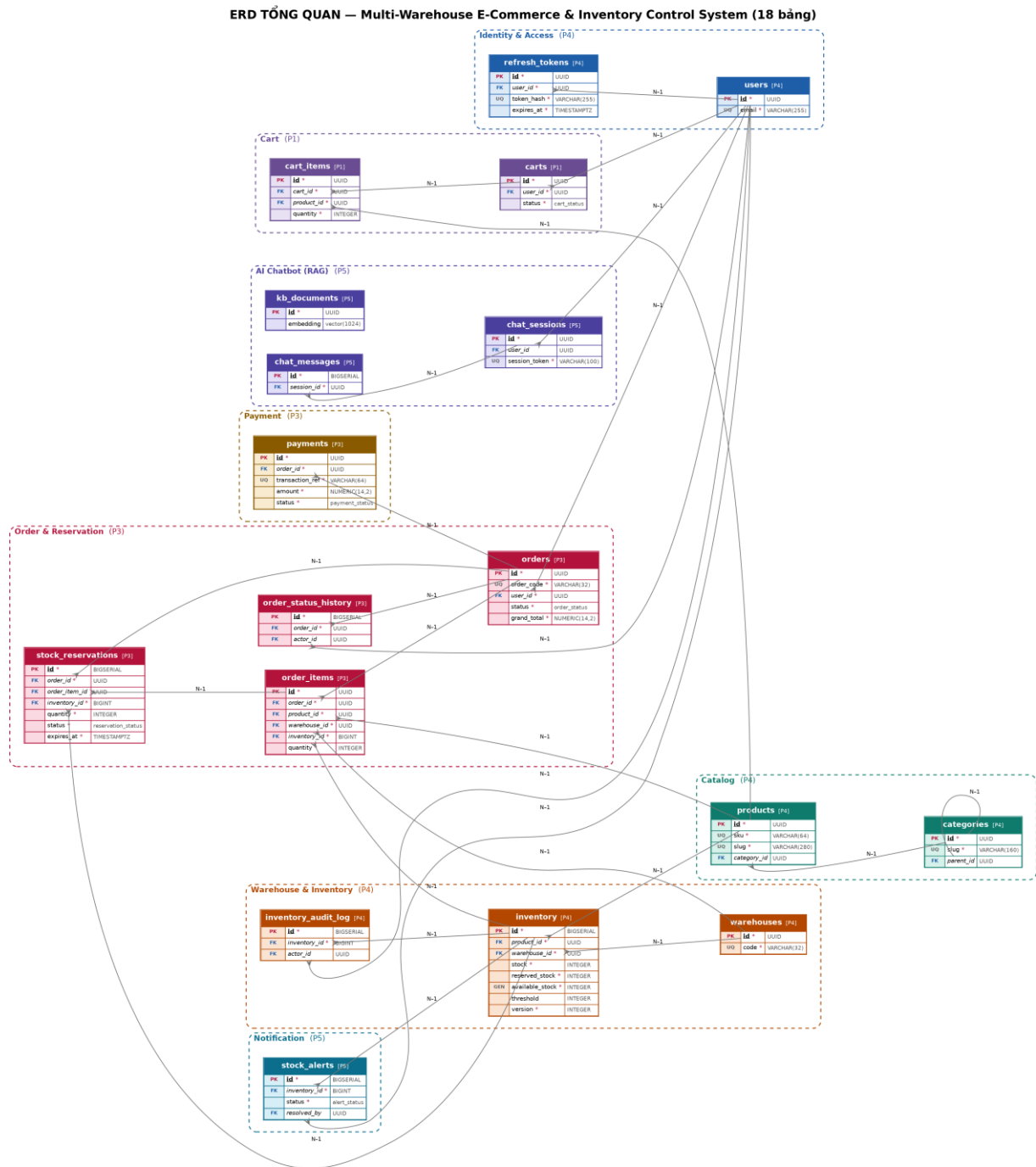
Index	Phục vụ truy vấn nào	Truy vết
<code>idx_res_cleanup (partial)</code>	Cron job mỗi phút quét reservation quá hạn. Là partial index chỉ chứa dòng <code>status='HOLDING'</code> nên rất nhỏ, quét cực nhanh dù bảng có hàng triệu bản ghi lịch sử.	<i>FR-ORD-04, NFR-04.3</i>
<code>idx_inventory_available (partial)</code>	Lọc sản phẩm còn hàng trên storefront. Chỉ chứa dòng <code>available_stock > 0</code> .	<i>FR-CAT-01</i>
<code>idx_inventory_lowstock</code>	Quét mặt hàng dưới ngưỡng theo từng kho cho dashboard và cảnh báo.	<i>FR-NOT-01, FR-DASH-01</i>
<code>idx_orders_user_time</code>	Lấy lịch sử đơn của chính người dùng, sắp xếp mới nhất trước.	<i>FR-ORD-03</i>
<code>idx_orders_status_time</code>	Tổng hợp doanh thu và số đơn theo khoảng thời gian cho báo cáo.	<i>FR-DASH-02</i>
<code>idx_products_name_trgm (GIN)</code>	Tìm kiếm gần đúng theo tên sản phẩm; dùng extension <code>pg_trgm</code> nên tìm được cả khi gõ thiếu dấu hoặc sai chính tả nhẹ.	<i>FR-CAT-01, NFR-01.2</i>
<code>idx_kb_embedding_hnsw</code>	Tìm kiếm ngữ nghĩa trong tài liệu chính sách. Chọn HNSW thay vì IVFFlat vì ở quy mô vài trăm đoạn văn bản, HNSW cho độ chính xác cao hơn và không cần huấn luyện trước.	<i>FR-BOT-01</i>
<code>idx_audit_reference</code>	Truy vết mọi biến động kho theo một đơn hàng cụ thể khi đối soát.	<i>NFR-03.1, NFR-08.2</i>

8. SƠ ĐỒ THỰC THỂ – LIÊN KẾT (ERD)

Ba biến thể ERD được cung cấp cho ba mục đích đọc khác nhau. Quy ước ký hiệu chung: gạch chân đậm là khóa chính, chữ nghiêng là khóa ngoại, dấu sao đỏ (*) nghĩa là cột NOT NULL, ký hiệu chân quạ ở đầu đường nối biểu thị phía “nhiều”. Màu khung thể hiện module sở hữu.

8.1. ERD tổng quan toàn hệ thống

Hiển thị đủ 18 bảng với khóa chính, khóa ngoại và các cột nghiệp vụ trọng yếu, nhóm theo tám module. Dùng để nhìn bức tranh tổng thể và xác định ranh giới sở hữu giữa P1–P5.



Hình 1. ERD tổng quan — 18 bảng, 25 quan hệ, phân theo 8 module

Phóng to chín bảng tham gia trực tiếp vào luồng đặt hàng và trừ kho, hiển thị đầy đủ mọi cột. Các đường nổi màu đỏ đậm là đường đi của dữ liệu trong Stock Reservation Engine. Đây là sơ đồ cần trình bày kỹ nhất khi bảo vệ đồ án.

payments (P3)

PK	id *	UUID
FK	order_id * <th>UUID</th>	UUID
UQ	transaction_id * <th>VARCHAR(64)</th>	VARCHAR(64)
	amount * <th>NUMERIC(14,2)</th>	NUMERIC(14,2)
	currency * <th>CHAR(3)</th>	CHAR(3)
	method * <th>payment_method</th>	payment_method
	status * <th>payment_status</th>	payment_status
	failure_reason <th>VARCHAR(255)</th>	VARCHAR(255)
	gateway_response <th>JSONB</th>	JSONB
	callback_count * <th>INTEGER</th>	INTEGER
	processed_at <th>TIMESTAMPZ</th>	TIMESTAMPZ
	created_at * <th>TIMESTAMPZ</th>	TIMESTAMPZ
	updated_at * <th>TIMESTAMPZ</th>	TIMESTAMPZ

Order & Reservation (P3)

stock_reservations (P3)

PK	id *	BIGSERIAL
FK	order_id * <th>UUID</th>	UUID
FK	order_item_id * <th>UUID</th>	UUID
	quantity * <th>INTEGER</th>	INTEGER
	status <th>reservation_status</th>	reservation_status
	expires_at * <th>TIMESTAMPZ</th>	TIMESTAMPZ
	committed_at <th>TIMESTAMPZ</th>	TIMESTAMPZ
	released_at * <th>TIMESTAMPZ</th>	TIMESTAMPZ
	created_at * <th>TIMESTAMPZ</th>	TIMESTAMPZ
	updated_at * <th>TIMESTAMPZ</th>	TIMESTAMPZ

order_items (P3)

PK	id *	UUID
FK	order_id * <th>UUID</th>	UUID
FK	product_id * <th>UUID</th>	UUID
FK	warehouse_id * <th>UUID</th>	UUID
FK	inventory_id * <th>BIGINT</th>	BIGINT
	product_name * <th>VARCHAR(255)</th>	VARCHAR(255)
	sku * <th>VARCHAR(64)</th>	VARCHAR(64)
	unit_price * <th>NUMERIC(12,2)</th>	NUMERIC(12,2)
	discount_amount * <th>NUMERIC(14,2)</th>	NUMERIC(14,2)
	quantity * <th>INTEGER</th>	INTEGER
	line_total * <th>NUMERIC(14,2)</th>	NUMERIC(14,2)
	created_at * <th>TIMESTAMPZ</th>	TIMESTAMPZ

orders (P3)

PK	id *	UUID
UQ	order_code * <th>VARCHAR(32)</th>	VARCHAR(32)
FK	user_id * <th>UUID</th>	UUID
	status * <th>order_status</th>	order_status
	subtotal * <th>NUMERIC(14,2)</th>	NUMERIC(14,2)
	discount_total * <th>NUMERIC(14,2)</th>	NUMERIC(14,2)
	shipping_fee * <th>NUMERIC(14,2)</th>	NUMERIC(14,2)
	grand_total * <th>NUMERIC(14,2)</th>	NUMERIC(14,2)
	currency * <th>CHAR(3)</th>	CHAR(3)
	shipping_recipient * <th>VARCHAR(150)</th>	VARCHAR(150)
	shipping_phone * <th>VARCHAR(20)</th>	VARCHAR(20)
	shipping_address * <th>VARCHAR(500)</th>	VARCHAR(500)
	shipping_region <th>VARCHAR(100)</th>	VARCHAR(100)
	reservation_expires_at <th>TIMESTAMPZ</th>	TIMESTAMPZ
	cancellation_reason <th>VARCHAR(255)</th>	VARCHAR(255)
	confirmed_at <th>TIMESTAMPZ</th>	TIMESTAMPZ
	cancelled_at <th>TIMESTAMPZ</th>	TIMESTAMPZ
	created_at * <th>TIMESTAMPZ</th>	TIMESTAMPZ
	updated_at * <th>TIMESTAMPZ</th>	TIMESTAMPZ

Warehouse & Inventory (P4)

inventory_audit_log (P4)

PK	id *	BIGSERIAL
FK	inventory_id * <th>BIGINT</th>	BIGINT
	reason * <th>inventory_change_reason</th>	inventory_change_reason
	stock_before * <th>INTEGER</th>	INTEGER
	stock_after * <th>INTEGER</th>	INTEGER
	reserved_before * <th>INTEGER</th>	INTEGER
	reserved_after * <th>INTEGER</th>	INTEGER
	quantity_delta * <th>INTEGER</th>	INTEGER
	reference_type <th>VARCHAR(40)</th>	VARCHAR(40)
	reference_id <th>VARCHAR(64)</th>	VARCHAR(64)
FK	actor_id * <th>UUID</th>	UUID
	note <th>VARCHAR(500)</th>	VARCHAR(500)
	created_at * <th>TIMESTAMPZ</th>	TIMESTAMPZ

inventory (P4)

PK	id *	BIGSERIAL
FK	product_id * <th>UUID</th>	UUID
FK	warehouse_id * <th>UUID</th>	UUID
	stock * <th>INTEGER</th>	INTEGER
	reserved_stock * <th>INTEGER</th>	INTEGER
	available_stock * <th>INTEGER</th>	INTEGER
	threshold <th>INTEGER</th>	INTEGER
	version * <th>INTEGER</th>	INTEGER
	last_restocked_at <th>TIMESTAMPZ</th>	TIMESTAMPZ
	created_at * <th>TIMESTAMPZ</th>	TIMESTAMPZ
	updated_at * <th>TIMESTAMPZ</th>	TIMESTAMPZ

warehouses (P4)

PK	id *	UUID
UQ	code * <th>VARCHAR(32)</th>	VARCHAR(32)
	name * <th>VARCHAR(150)</th>	VARCHAR(150)
	region * <th>VARCHAR(100)</th>	VARCHAR(100)
	address <th>VARCHAR(500)</th>	VARCHAR(500)
	capacity <th>INTEGER</th>	INTEGER
	default_low_stock_threshold * <th>INTEGER</th>	INTEGER
	is_active * <th>BOOLEAN</th>	BOOLEAN
	created_at * <th>TIMESTAMPZ</th>	TIMESTAMPZ
	updated_at * <th>TIMESTAMPZ</th>	TIMESTAMPZ

Catalog (P4)

products (P4)

PK	id *	UUID
UQ	sku * <th>VARCHAR(64)</th>	VARCHAR(64)
	name * <th>VARCHAR(255)</th>	VARCHAR(255)
UQ	slug * <th>VARCHAR(280)</th>	VARCHAR(280)
	description <th>TEXT</th>	TEXT
	base_price * <th>NUMERIC(14,2)</th>	NUMERIC(14,2)
	discount_percent * <th>NUMERIC(5,2)</th>	NUMERIC(5,2)
FK	category_id * <th>UUID</th>	UUID
	image_url <th>VARCHAR(500)</th>	VARCHAR(500)
	is_active * <th>BOOLEAN</th>	BOOLEAN
	created_at * <th>TIMESTAMPZ</th>	TIMESTAMPZ
	updated_at * <th>TIMESTAMPZ</th>	TIMESTAMPZ

Notification (P5)

stock_alerts (P5)

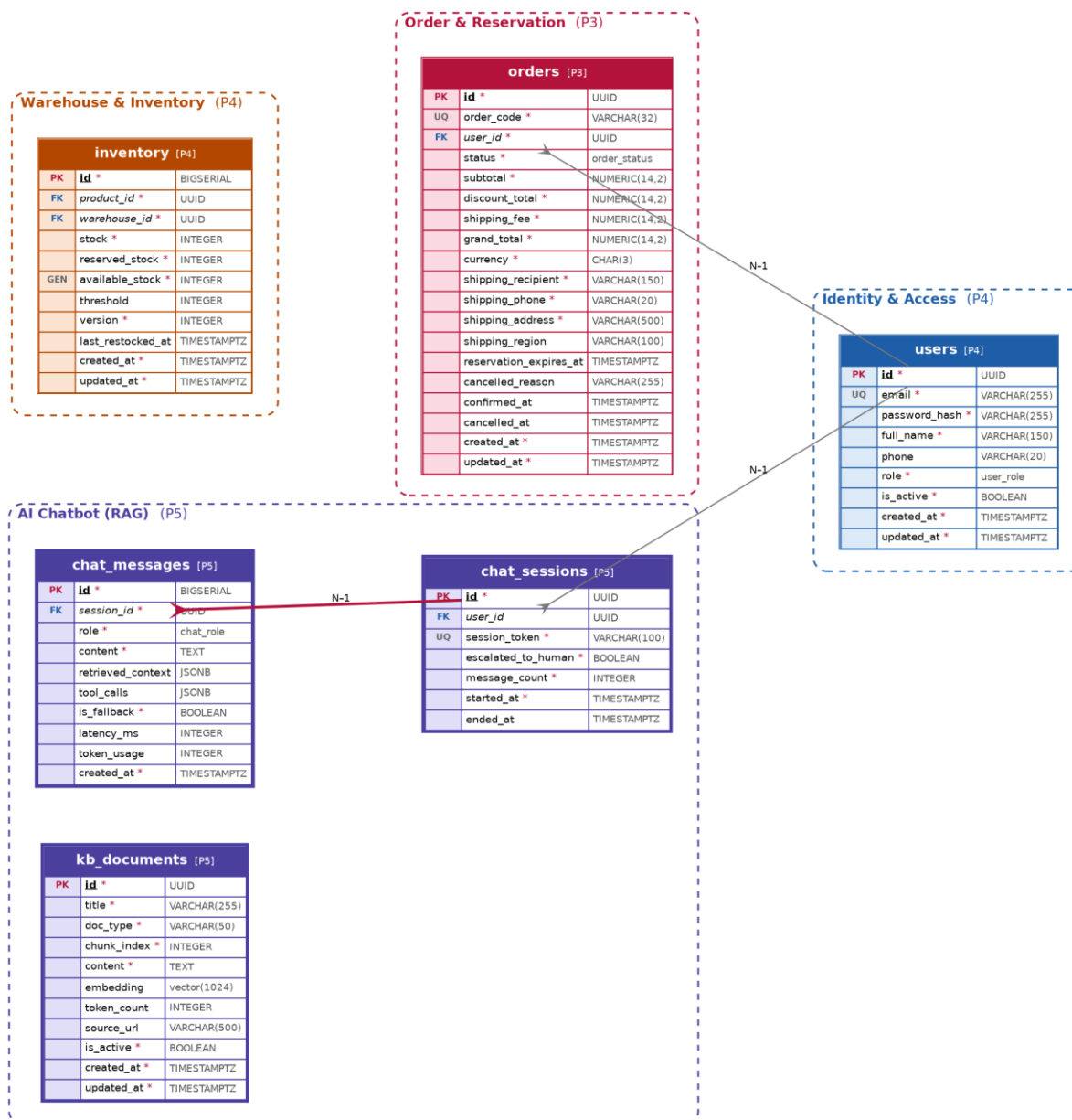
PK	id *	BIGSERIAL
FK	inventory_id * <th>BIGINT</th>	BIGINT
	alert_level * <th>alert_level</th>	alert_level
	status * <th>alert_status</th>	alert_status
	stock_at_trigger * <th>INTEGER</th>	INTEGER
	available_at_trigger * <th>INTEGER</th>	INTEGER
	threshold_at_trigger * <th>INTEGER</th>	INTEGER
	triggered_at * <th>TIMESTAMPZ</th>	TIMESTAMPZ
	resolved_at <th>TIMESTAMPZ</th>	TIMESTAMPZ
FK	resolved_by * <th>UUID</th>	UUID

Thiết kế CSDL & ERD — Topic 3 · Course 502045 | Trang 25 / 42

8.3. ERD module AI Chatbot (RAG)

Thể hiện hai nguồn dữ liệu của kiến trúc RAG: nguồn tĩnh là kb_documents đã vector hóa, và nguồn động là orders cùng inventory được truy vấn qua function calling. Hai nguồn này không nối trực tiếp với nhau bằng khóa ngoại — chúng được ghép lại ở tầng ứng dụng khi xây dựng prompt, và kết quả ghép được lưu vào chat_messages.retrieved_context để kiểm chứng về sau.

ERD MODULE AI CHATBOT (RAG) — pgvector + Function Calling



Hình 3. ERD module AI Chatbot — pgvector và function calling

9. KẾT QUẢ KIỂM CHỨNG THỰC TẾ

Toàn bộ thiết kế đã được thực thi thật trên PostgreSQL 16.15 để xác nhận tính đúng đắn trước khi bàn giao, thay vì chỉ dừng ở mức sơ đồ trên giấy.

9.1. Kết quả tạo schema

Đối tượng	Số lượng tạo thành công
Bảng (BASE TABLE)	18
Kiểu ENUM	10
Index	62
Ràng buộc CHECK	21
Khóa ngoại	25
Trigger tự cập nhật updated_at	11
Lỗi cú pháp / lỗi thực thi	0

9.2. Kiểm thử tình huống hai khách tranh sản phẩm cuối cùng

Kịch bản: sản phẩm SKU-1001 tại kho WH-HCM chỉ còn 1 đơn vị. Hai phiên giao dịch chạy song song, cùng đặt mua 1 đơn vị. Đây chính là Scenario trong User Story US-301 của kế hoạch.

Bước	Khách A	Khách B	Kết quả
Khóa dòng tồn kho	SELECT ... FOR UPDATE	Phải CHỜ (bị chặn)	Chỉ một phiên vào được vùng tới hạn
Kiểm tra tồn khả dụng	available = 1 $\geq$ 1 $\rightarrow$ hợp lệ	available = 0 < 1 $\rightarrow$ không đủ	Khách B thấy đúng dữ liệu sau khi A commit
Kết quả giao dịch	COMMIT thành công	Ném INSUFFICIENT_STOCK $\rightarrow$ ROLLBACK	Đúng 1 đơn được tạo
Trạng thái cuối	reservation HOLDING	Đơn hàng bị hủy hoàn toàn	reserved_stock = 1, tổng giữ chỗ = 1 — nhất quán

Lỗi nghiêm trọng phát hiện được trong quá trình kiểm thử — P3 bắt buộc phải đọc

Ở lần chạy thử đầu tiên, thiết kế dùng cách viết trực giác nhất:

```
UPDATE inventory SET reserved_stock = reserved_stock + 1 WHERE id = ? AND available_stock >= 1;
```

rồi chèn thẳng bản ghi reservation. Kết quả đo được: reserved_stock = 1 (đúng) nhưng tồn tại 2 bản ghi reservation HOLDING (sai).

Nguyên nhân: khi không đủ hàng, câu UPDATE khớp 0 dòng nhưng KHÔNG ném lỗi. Chương trình vẫn chạy tiếp và tạo ra một reservation “ma” — khách hàng nhận thông báo đặt hàng thành công trong khi hệ thống chưa hề giữ hàng cho họ. Đây là oversell ở dạng âm thầm, và không một CHECK constraint nào bắt được vì bản thân reserved_stock vẫn hợp lệ.

Cách khắc phục đã kiểm chứng: sau khi UPDATE, bắt buộc kiểm tra số dòng bị ảnh hưởng; bằng 0 thì phải ném ConflictException để rollback toàn bộ transaction. Hàm mẫu fn_reserve_stock ở mục 10.2 hiện thực đúng cách này và đã chạy qua kiểm thử song song.

9.3. Kiểm thử cron job giải phóng reservation hết hạn

Phép kiểm tra	Kỳ vọng	Kết quả đo được
Giải phóng reserved_stock	1 → 0	1 → 0 — đạt
Chuyển trạng thái reservation	HOLDING → EXPIRED	EXPIRED — đạt
Chuyển trạng thái đơn hàng	PENDING_PAYMENT → CANCELLED, lý do EXPIRED	CANCELLED / EXPIRED — đạt
Tính idempotent (chạy lần 2)	Không trừ reserved_stock thêm lần nữa	UPDATE 0 dòng, reserved_stock giữ nguyên 0 — đạt
Index được sử dụng	Dùng idx_res_cleanup	Index Scan using idx_res_cleanup — đạt

9.4. Một ràng buộc đã bị loại bỏ sau khi kiểm thử

Phiên bản đầu của thiết kế có ràng buộc CHECK (expires_at > created_at) trên bảng stock_reservations. Khi chạy thử kịch bản kiểm thử của NFR-04.3, ràng buộc này chặn luôn việc lùi expires_at về quá khứ — thao tác bắt buộc phải làm để mô phỏng reservation hết hạn trong integration test.

Ràng buộc đã được gỡ bỏ. Giá trị bảo vệ của nó rất thấp (expires_at luôn do server sinh ra, không nhận từ client) trong khi nó làm cả nhóm không viết được test fixture cho một NFR bắt buộc. Đây là ví dụ cho thấy ràng buộc CSDL cần được cân nhắc cùng với nhu cầu kiểm thử, không chỉ nhìn từ góc độ đúng sai nghiệp vụ.

10. MÃ NGUỒN DDL (PostgreSQL)

10.1. Script tạo toàn bộ schema

Script dưới đây đã chạy thành công không lỗi trên PostgreSQL 16.15 với cờ ON_ERROR_STOP=1. P4 có thể dùng trực tiếp, hoặc dùng schema Prisma ở mục 11 rồi bổ sung phần cột sinh và partial index bằng migration SQL tay.

```
-- =====
-- Multi-Warehouse E-Commerce & Inventory Control System
-- Topic 3 - Course ID 502045
-- PostgreSQL 16+ / DDL day du (18 bang, 10 ENUM)
-- =====

-- ----- 0. EXTENSIONS -----
CREATE EXTENSION IF NOT EXISTS "pgcrypto"; -- gen_random_uuid()
CREATE EXTENSION IF NOT EXISTS "vector"; -- pgvector cho AI Chatbot RAG
CREATE EXTENSION IF NOT EXISTS "pg_trgm"; -- tăng tốc tìm kiếm tên sản phẩm (FR-CAT-01)

-- ----- 1. ENUM TYPES -----
CREATE TYPE user_role AS ENUM ('CUSTOMER', 'WAREHOUSE_STAFF', 'ADMIN');
CREATE TYPE order_status AS ENUM ('PENDING_PAYMENT', 'CONFIRMED', 'SHIPPING', 'DELIVERED', 'CANCELLED');
CREATE TYPE reservation_status AS ENUM ('HOLDING', 'COMMITTED', 'RELEASED', 'EXPIRED');
CREATE TYPE payment_status AS ENUM ('PENDING', 'SUCCESS', 'FAILED');
CREATE TYPE payment_method AS ENUM ('MOCK_CARD', 'MOCK_WALLET', 'COD');
CREATE TYPE alert_level AS ENUM ('WARNING', 'CRITICAL');
CREATE TYPE alert_status AS ENUM ('OPEN', 'RESOLVED');
CREATE TYPE inventory_change_reason AS ENUM ('RESERVE', 'COMMIT', 'RELEASE', 'EXPIRE', 'RESTOCK', 'MANUAL_ADJUST');
CREATE TYPE chat_role AS ENUM ('USER', 'ASSISTANT', 'SYSTEM');
CREATE TYPE cart_status AS ENUM ('ACTIVE', 'CONVERTED', 'ABANDONED');

-- =====
-- NHOM A: IDENTITY & ACCESS (P4)
-- =====
CREATE TABLE users (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    email VARCHAR(255) NOT NULL UNIQUE,
    password_hash VARCHAR(255) NOT NULL,
    full_name VARCHAR(150) NOT NULL,
    phone VARCHAR(20),
    role user_role NOT NULL DEFAULT 'CUSTOMER',
    is_active BOOLEAN NOT NULL DEFAULT TRUE,
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    CONSTRAINT chk_users_email_format CHECK (email ~* '^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}$')
);
CREATE INDEX idx_users_role ON users(role) WHERE is_active = TRUE;

CREATE TABLE refresh_tokens (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    token_hash VARCHAR(255) NOT NULL UNIQUE,
    expires_at TIMESTAMPTZ NOT NULL,
    revoked_at TIMESTAMPTZ,
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
CREATE INDEX idx_refresh_user_active ON refresh_tokens(user_id) WHERE revoked_at IS NULL;

-- =====
-- NHOM B: CATALOG (P4)
-- =====
CREATE TABLE categories (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    name VARCHAR(150) NOT NULL,
    slug VARCHAR(160) NOT NULL UNIQUE,
    parent_id UUID REFERENCES categories(id) ON DELETE SET NULL,
    is_active BOOLEAN NOT NULL DEFAULT TRUE,
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    CONSTRAINT chk_categories_no_self_parent CHECK (parent_id IS NULL OR parent_id <> id)
);
CREATE INDEX idx_categories_parent ON categories(parent_id);

CREATE TABLE products (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    sku VARCHAR(64) NOT NULL UNIQUE,
    name VARCHAR(255) NOT NULL,
    slug VARCHAR(280) NOT NULL UNIQUE,
    description TEXT,
    base_price NUMERIC(14,2) NOT NULL,
    discount_percent NUMERIC(5,2) NOT NULL DEFAULT 0,
    category_id UUID REFERENCES categories(id) ON DELETE SET NULL,
    image_url VARCHAR(500),
    is_active BOOLEAN NOT NULL DEFAULT TRUE,
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    CONSTRAINT chk_products_price_positive CHECK (base_price >= 0),
    CONSTRAINT chk_products_discount_range CHECK (discount_percent >= 0 AND discount_percent <= 100)
);
```

```

CREATE INDEX idx_products_category ON products(category_id) WHERE is_active = TRUE;
CREATE INDEX idx_products_price ON products(base_price) WHERE is_active = TRUE;
CREATE INDEX idx_products_name_trgm ON products USING gin (name gin_trgm_ops);

-- =====
-- NHOM C: WAREHOUSE & INVENTORY (P4) -- LOI KY THUAT
-- =====
CREATE TABLE warehouses (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    code VARCHAR(32) NOT NULL UNIQUE,
    name VARCHAR(150) NOT NULL,
    region VARCHAR(100) NOT NULL,
    address VARCHAR(500),
    capacity INTEGER,
    default_low_stock_threshold INTEGER NOT NULL DEFAULT 10,
    is_active BOOLEAN NOT NULL DEFAULT TRUE,
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    CONSTRAINT chk_wh_threshold_nonneg CHECK (default_low_stock_threshold >= 0),
    CONSTRAINT chk_wh_capacity_pos CHECK (capacity IS NULL OR capacity > 0)
);
CREATE INDEX idx_warehouses_region ON warehouses(region) WHERE is_active = TRUE;

-- BANG TRUNG TAM: hien thuc quan he N-N products <-> warehouses
CREATE TABLE inventory (
    id BIGSERIAL PRIMARY KEY,
    product_id UUID NOT NULL REFERENCES products(id) ON DELETE CASCADE,
    warehouse_id UUID NOT NULL REFERENCES warehouses(id) ON DELETE RESTRICT,
    stock INTEGER NOT NULL DEFAULT 0,
    reserved_stock INTEGER NOT NULL DEFAULT 0,
    available_stock INTEGER GENERATED ALWAYS AS (stock - reserved_stock) STORED,
    threshold INTEGER,
    version INTEGER NOT NULL DEFAULT 0,
    last_restocked_at TIMESTAMPTZ,
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),

    -- RANG BUOC SONG CON: mot cap (san pham, kho) chi ton tai DUY NHAT 1 dong
    CONSTRAINT uq_inventory_product_warehouse UNIQUE (product_id, warehouse_id),

    -- 3 CHECK constraint chong oversell o tang CSDL (lop bao ve cuoi cung)
    CONSTRAINT chk_inv_stock_nonneg CHECK (stock >= 0),
    CONSTRAINT chk_inv_reserved_nonneg CHECK (reserved_stock >= 0),
    CONSTRAINT chk_inv_reserved_le_stock CHECK (reserved_stock <= stock),
    CONSTRAINT chk_inv_threshold_nonneg CHECK (threshold IS NULL OR threshold >= 0)
);
CREATE INDEX idx_inventory_product ON inventory(product_id);
CREATE INDEX idx_inventory_warehouse ON inventory(warehouse_id);
-- Phuc vu FR-CAT-01 (loc san pham con hang) va FR-NOT-01 (quet low-stock)
CREATE INDEX idx_inventory_available ON inventory(available_stock) WHERE available_stock > 0;
CREATE INDEX idx_inventory_lowstock ON inventory(warehouse_id, available_stock);

CREATE TABLE inventory_audit_log (
    id BIGSERIAL PRIMARY KEY,
    inventory_id BIGINT NOT NULL REFERENCES inventory(id) ON DELETE CASCADE,
    reason inventory_change_reason NOT NULL,
    stock_before INTEGER NOT NULL,
    stock_after INTEGER NOT NULL,
    reserved_before INTEGER NOT NULL,
    reserved_after INTEGER NOT NULL,
    quantity_delta INTEGER NOT NULL,
    reference_type VARCHAR(40),
    reference_id VARCHAR(64),
    actor_id UUID REFERENCES users(id) ON DELETE SET NULL,
    note VARCHAR(500),
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
CREATE INDEX idx_audit_inventory_time ON inventory_audit_log(inventory_id, created_at DESC);
CREATE INDEX idx_audit_reference ON inventory_audit_log(reference_type, reference_id);

-- =====
-- NHOM D: CART (P1) -- TUY CHON neu chot gio hang server-side
-- =====
CREATE TABLE carts (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    status cart_status NOT NULL DEFAULT 'ACTIVE',
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
-- Moi user chi co toi da 1 gio ACTIVE
CREATE UNIQUE INDEX uq_carts_one_active_per_user ON carts(user_id) WHERE status = 'ACTIVE';

CREATE TABLE cart_items (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    cart_id UUID NOT NULL REFERENCES carts(id) ON DELETE CASCADE,
    product_id UUID NOT NULL REFERENCES products(id) ON DELETE CASCADE,
    quantity INTEGER NOT NULL DEFAULT 1,
    added_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    CONSTRAINT uq_cart_items_cart_product UNIQUE (cart_id, product_id),
    CONSTRAINT chk_cart_items_qty_pos CHECK (quantity > 0)
);
-- =====

```

```

-- NHOM E: ORDER & RESERVATION (P3) -- LOI KY THUAT
-- =====
CREATE TABLE orders (
  id                UUID                PRIMARY KEY DEFAULT gen_random_uuid(),
  order_code        VARCHAR(32)         NOT NULL UNIQUE,
  user_id           UUID                NOT NULL REFERENCES users(id) ON DELETE RESTRICT,
  status            order_status        NOT NULL DEFAULT 'PENDING_PAYMENT',
  subtotal          NUMERIC(14,2)       NOT NULL DEFAULT 0,
  discount_total    NUMERIC(14,2)       NOT NULL DEFAULT 0,
  shipping_fee      NUMERIC(14,2)       NOT NULL DEFAULT 0,
  grand_total       NUMERIC(14,2)       NOT NULL DEFAULT 0,
  currency          CHAR(3)             NOT NULL DEFAULT 'VND',
  shipping_recipient VARCHAR(150)       NOT NULL,
  shipping_phone    VARCHAR(20)         NOT NULL,
  shipping_address  VARCHAR(500)        NOT NULL,
  shipping_region   VARCHAR(100),
  reservation_expires_at TIMESTAMPTZ,
  cancelled_reason  VARCHAR(255),
  confirmed_at      TIMESTAMPTZ,
  cancelled_at      TIMESTAMPTZ,
  created_at        TIMESTAMPTZ        NOT NULL DEFAULT NOW(),
  updated_at        TIMESTAMPTZ        NOT NULL DEFAULT NOW(),
  CONSTRAINT chk_orders_amounts_nonneg CHECK (
    subtotal >= 0 AND discount_total >= 0 AND shipping_fee >= 0 AND grand_total >= 0
  ),
  CONSTRAINT chk_orders_grand_total CHECK (
    grand_total = subtotal - discount_total + shipping_fee
  )
);
-- FR-ORD-03: lay don cua chinh user, moi nhat truoc
CREATE INDEX idx_orders_user_time ON orders(user_id, created_at DESC);
CREATE INDEX idx_orders_status ON orders(status);
-- FR-DASH-02: bao cao doanh thu theo khoang thoi gian
CREATE INDEX idx_orders_status_time ON orders(status, created_at);

CREATE TABLE order_items (
  id                UUID                PRIMARY KEY DEFAULT gen_random_uuid(),
  order_id          UUID                NOT NULL REFERENCES orders(id) ON DELETE CASCADE,
  product_id        UUID                NOT NULL REFERENCES products(id) ON DELETE RESTRICT,
  warehouse_id      UUID                NOT NULL REFERENCES warehouses(id) ON DELETE RESTRICT,
  inventory_id       BIGINT             NOT NULL REFERENCES inventory(id) ON DELETE RESTRICT,
  product_name      VARCHAR(255)        NOT NULL,
  sku               VARCHAR(64)         NOT NULL,
  unit_price        NUMERIC(14,2)       NOT NULL,
  discount_amount    NUMERIC(14,2)       NOT NULL DEFAULT 0,
  quantity          INTEGER             NOT NULL,
  line_total        NUMERIC(14,2)       NOT NULL,
  created_at        TIMESTAMPTZ        NOT NULL DEFAULT NOW(),
  CONSTRAINT chk_oi_qty_pos CHECK (quantity > 0),
  CONSTRAINT chk_oi_price_nonneg CHECK (unit_price >= 0 AND discount_amount >= 0),
  CONSTRAINT chk_oi_line_total CHECK (line_total = unit_price * quantity - discount_amount)
);
CREATE INDEX idx_order_items_order ON order_items(order_id);
CREATE INDEX idx_order_items_product ON order_items(product_id);
CREATE INDEX idx_order_items_warehouse ON order_items(warehouse_id);

-- BANG CHONG OVERSELL
CREATE TABLE stock_reservations (
  id                BIGSERIAL           PRIMARY KEY,
  order_id          UUID                NOT NULL REFERENCES orders(id) ON DELETE CASCADE,
  order_item_id     UUID                REFERENCES order_items(id) ON DELETE CASCADE,
  inventory_id       BIGINT             NOT NULL REFERENCES inventory(id) ON DELETE RESTRICT,
  quantity          INTEGER             NOT NULL,
  status            reservation_status NOT NULL DEFAULT 'HOLDING',
  expires_at        TIMESTAMPTZ        NOT NULL,
  committed_at      TIMESTAMPTZ,
  released_at       TIMESTAMPTZ,
  created_at        TIMESTAMPTZ        NOT NULL DEFAULT NOW(),
  updated_at        TIMESTAMPTZ        NOT NULL DEFAULT NOW(),
  CONSTRAINT chk_res_qty_pos CHECK (quantity > 0)
  -- KHONG dat CHECK (expires_at > created_at): se chan test fixture cua NFR-04.3
  -- (can lui expires_at ve qua khu de mo phong reservation het han)
);
-- INDEX QUAN TRONG NHAT cho cron job FR-ORD-04 (quet reservation het han moi phut)
CREATE INDEX idx_res_cleanup ON stock_reservations(expires_at)
  WHERE status = 'HOLDING';
CREATE INDEX idx_res_order ON stock_reservations(order_id);
CREATE INDEX idx_res_inventory ON stock_reservations(inventory_id, status);

CREATE TABLE order_status_history (
  id                BIGSERIAL           PRIMARY KEY,
  order_id          UUID                NOT NULL REFERENCES orders(id) ON DELETE CASCADE,
  from_status       order_status,
  to_status         order_status        NOT NULL,
  actor_id          UUID                REFERENCES users(id) ON DELETE SET NULL,
  reason            VARCHAR(255),
  created_at        TIMESTAMPTZ        NOT NULL DEFAULT NOW(),
  CONSTRAINT chk_osh_status_changed CHECK (from_status IS NULL OR from_status <> to_status)
);
CREATE INDEX idx_osh_order_time ON order_status_history(order_id, created_at);

-- =====
-- NHOM F: PAYMENT (P3)
-- =====
CREATE TABLE payments (
  id                UUID                PRIMARY KEY DEFAULT gen_random_uuid(),
  order_id          UUID                NOT NULL REFERENCES orders(id) ON DELETE RESTRICT,

```

```

-- KHOA IDEMPOTENCY: UNIQUE bao dam callback lap khong the tao giao dich thu 2
transaction_ref VARCHAR(64) NOT NULL UNIQUE,
amount NUMERIC(14,2) NOT NULL,
currency CHAR(3) NOT NULL DEFAULT 'VND',
method payment_method NOT NULL,
status payment_status NOT NULL DEFAULT 'PENDING',
failure_reason VARCHAR(255),
gateway_response JSONB,
callback_count INTEGER NOT NULL DEFAULT 0,
processed_at TIMESTAMPTZ,
created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
CONSTRAINT chk_pay_amount_pos CHECK (amount > 0),
CONSTRAINT chk_pay_callback_count CHECK (callback_count >= 0)
);
CREATE INDEX idx_payments_order ON payments(order_id);
CREATE INDEX idx_payments_status ON payments(status);
-- Moi don chi co TOI DA 1 giao dich SUCCESS -> chan double-charge o tang CSDL
CREATE UNIQUE INDEX uq_payments_one_success_per_order
ON payments(order_id) WHERE status = 'SUCCESS';

-- =====
-- NHOM G: NOTIFICATION (P5)
-- =====
CREATE TABLE stock_alerts (
  id BIGSERIAL PRIMARY KEY,
  inventory_id BIGINT NOT NULL REFERENCES inventory(id) ON DELETE CASCADE,
  alert_level alert_level NOT NULL DEFAULT 'WARNING',
  status alert_status NOT NULL DEFAULT 'OPEN',
  stock_at_trigger INTEGER NOT NULL,
  available_at_trigger INTEGER NOT NULL,
  threshold_at_trigger INTEGER NOT NULL,
  triggered_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  resolved_at TIMESTAMPTZ,
  resolved_by UUID REFERENCES users(id) ON DELETE SET NULL
);
CREATE INDEX idx_alerts_open ON stock_alerts(triggered_at DESC) WHERE status = 'OPEN';
-- Chong spam: moi dong ton kho chi co toi da 1 canh bao OPEN cung luc (FR-NOT-01)
CREATE UNIQUE INDEX uq_alerts_one_open_per_inventory
ON stock_alerts(inventory_id) WHERE status = 'OPEN';

-- =====
-- NHOM H: AI CHATBOT (RAG) (P5)
-- =====
CREATE TABLE kb_documents (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  title VARCHAR(255) NOT NULL,
  doc_type VARCHAR(50) NOT NULL,
  chunk_index INTEGER NOT NULL DEFAULT 0,
  content TEXT NOT NULL,
  embedding vector(1024),
  token_count INTEGER,
  source_url VARCHAR(500),
  is_active BOOLEAN NOT NULL DEFAULT TRUE,
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
-- Index vector cho semantic search (cosine distance).
-- HNSW cho chat luong/toc do truy van tot hon IVFFlat o quy mo nho cua do an.
CREATE INDEX idx_kb_embedding_hnsw ON kb_documents
USING hnsw (embedding vector_cosine_ops);
CREATE INDEX idx_kb_doctype ON kb_documents(doc_type) WHERE is_active = TRUE;

CREATE TABLE chat_sessions (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID REFERENCES users(id) ON DELETE SET NULL,
  session_token VARCHAR(100) NOT NULL UNIQUE,
  escalated_to_human BOOLEAN NOT NULL DEFAULT FALSE,
  message_count INTEGER NOT NULL DEFAULT 0,
  started_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  ended_at TIMESTAMPTZ
);
CREATE INDEX idx_chat_sessions_user ON chat_sessions(user_id, started_at DESC);

CREATE TABLE chat_messages (
  id BIGSERIAL PRIMARY KEY,
  session_id UUID NOT NULL REFERENCES chat_sessions(id) ON DELETE CASCADE,
  role chat_role NOT NULL,
  content TEXT NOT NULL,
  retrieved_context JSONB,
  tool_calls JSONB,
  is_fallback BOOLEAN NOT NULL DEFAULT FALSE,
  latency_ms INTEGER,
  token_usage INTEGER,
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
CREATE INDEX idx_chat_messages_session ON chat_messages(session_id, created_at);

-- =====
-- TRIGGER: tu dong cap nhat updated_at
-- =====
CREATE OR REPLACE FUNCTION trg_set_updated_at()
RETURNS TRIGGER AS $$
BEGIN
  NEW.updated_at = NOW();
  RETURN NEW;

```

```

END;
$$ LANGUAGE plpgsql;

DO $$
DECLARE t TEXT;
BEGIN
    FOREACH t IN ARRAY ARRAY[
        'users','categories','products','warehouses','inventory',
        'carts','cart_items','orders','stock_reservations','payments',
        'kb_documents'
    ]
    LOOP
        EXECUTE format(
            'CREATE TRIGGER set_updated_at BEFORE UPDATE ON %I
            FOR EACH ROW EXECUTE FUNCTION trg_set_updated_at();', t);
    END LOOP;
END $$;

-- =====
-- TRIGGER: tang version moi khi inventory bi cap nhat
-- (ho tro optimistic locking du phong - defense in depth)
-- =====
CREATE OR REPLACE FUNCTION trg_inventory_bump_version()
RETURNS TRIGGER AS $$
BEGIN
    NEW.version = OLD.version + 1;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER inventory_bump_version
BEFORE UPDATE OF stock, reserved_stock ON inventory
FOR EACH ROW EXECUTE FUNCTION trg_inventory_bump_version();

```

10.2. Hàm giữ chỗ tồn kho mẫu (bản cài đặt đúng)

Hàm này hiện thực đúng luồng bảy bước của FR-ORD-02 và đã vượt qua kiểm thử song song ở mục 9.2. P3 có thể gọi trực tiếp từ Prisma bằng \$queryRaw, hoặc dùng làm tài liệu tham chiếu để viết lại bằng TypeScript trong StockReservationService.

```
-- =====
-- HAM GIU CHO TON KHO - ban cai dat DUNG cua FR-ORD-02
-- Da kiem chung bang test song song tren PostgreSQL 16.15
-- =====
CREATE OR REPLACE FUNCTION fn_reserve_stock(
  p_inventory_id BIGINT,
  p_order_id     UUID,
  p_quantity     INTEGER,
  p_ttl_minutes  INTEGER DEFAULT 15
) RETURNS BIGINT AS $$
DECLARE
  v_available INTEGER;
  v_res_id    BIGINT;
BEGIN
  -- B1: KHOA DONG (pessimistic lock).
  -- Moi transaction khac cham toi dong nay se phai CHO tai day.
  -- Caller BAT BUOC goi ham nay theo thu tu inventory_id TANG DAN
  -- khi mot don hang co nhieu san pham -> khong bao gio deadlock.
  SELECT available_stock INTO v_available
    FROM inventory
   WHERE id = p_inventory_id
   FOR UPDATE;

  IF NOT FOUND THEN
    RAISE EXCEPTION 'INVENTORY_NOT_FOUND' USING ERRCODE = 'P0002';
  END IF;

  -- B2: KIEM TRA DU HANG.
  -- Khong du -> NEM LOI de PostgreSQL rollback TOAN BO transaction.
  -- Day la buoc ma cach viet truc giac hay bo sot, sinh ra "reservation ma".
  IF v_available < p_quantity THEN
    RAISE EXCEPTION 'INSUFFICIENT_STOCK: con % can %', v_available, p_quantity
      USING ERRCODE = '23514'; -- NestJS map sang 409 Conflict
  END IF;

  -- B3: Tang reserved_stock (KHONG tru stock that o buoc nay)
  UPDATE inventory
   SET reserved_stock = reserved_stock + p_quantity
   WHERE id = p_inventory_id;

  -- B4: Tao ban ghi giu cho co thoi han
  INSERT INTO stock_reservations (order_id, inventory_id, quantity, status, expires_at)
  VALUES (p_order_id, p_inventory_id, p_quantity, 'HOLDING',
    NOW() + make_interval(mins => p_ttl_minutes))
  RETURNING id INTO v_res_id;

  RETURN v_res_id;
END;
$$ LANGUAGE plpgsql;

-- =====
-- LOP BAO VE CUOI CUNG: trigger kiem tra bat bien
-- Bat truong hop tong reservation HOLDING > reserved_stock
-- (chinh la loi "reservation ma" da phat hien o muc 9.2)
-- DEFERRABLE INITIALLY DEFERRED -> chi kiem tra o thoi diem COMMIT,
-- nen khong can tro cac buoc trung gian trong cung transaction.
-- =====
CREATE OR REPLACE FUNCTION trg_check_reservation_consistency()
RETURNS TRIGGER AS $$
DECLARE
  v_sum_holding INTEGER;
  v_reserved    INTEGER;
BEGIN
  SELECT COALESCE(SUM(quantity), 0) INTO v_sum_holding
    FROM stock_reservations
   WHERE inventory_id = NEW.inventory_id AND status = 'HOLDING';

  SELECT reserved_stock INTO v_reserved
    FROM inventory WHERE id = NEW.inventory_id;

  IF v_sum_holding > v_reserved THEN
    RAISE EXCEPTION
      'RESERVATION_INCONSISTENT: tong giu cho (%) > reserved_stock (%) tren inventory_id%',
      v_sum_holding, v_reserved, NEW.inventory_id
      USING ERRCODE = '23514';
  END IF;
  RETURN NULL;
END;
$$ LANGUAGE plpgsql;

CREATE CONSTRAINT TRIGGER ctrg_reservation_consistency
  AFTER INSERT OR UPDATE ON stock_reservations
  DEFERRABLE INITIALLY DEFERRED
```

```

FOR EACH ROW EXECUTE FUNCTION trg_check_reservation_consistency();

-- =====
-- CRON JOB FR-ORD-04: giai phong reservation qua han
-- Chay moi phut. SKIP LOCKED cho phep nhieu instance chay song song
-- ma khong xu ly trung cung mot ban ghi.
-- Da kiem chung: idempotent, chay lan 2 khong tru reserved_stock them.
-- =====
WITH expired AS (
  SELECT id, inventory_id, quantity, order_id
    FROM stock_reservations
   WHERE status = 'HOLDING'
     AND expires_at < NOW()
  ORDER BY inventory_id          -- khoa theo thu tu tang dan -> chong deadlock
  FOR UPDATE SKIP LOCKED
  LIMIT 100
), released AS (
  UPDATE stock_reservations sr
    SET status = 'EXPIRED', released_at = NOW()
    FROM expired e
   WHERE sr.id = e.id
  RETURNING sr.inventory_id, sr.quantity, sr.order_id
), restored AS (
  UPDATE inventory i
    SET reserved_stock = i.reserved_stock - r.quantity
    FROM released r
   WHERE i.id = r.inventory_id
  RETURNING i.id
)
UPDATE orders o
  SET status = 'CANCELLED', cancelled_reason = 'EXPIRED', cancelled_at = NOW()
  FROM released r
 WHERE o.id = r.order_id
    AND o.status = 'PENDING_PAYMENT';

```

11. SCHEMA PRISMA

File này tương ứng với Backend/prisma/schema.prisma — sản phẩm bàn giao của P4 và là điều kiện tiên quyết để P1, P2, P3 bắt đầu code song song.

Ba điểm Prisma không tự sinh được — P4 phải bổ sung bằng migration SQL tay

1. Cột sinh available_stock: Prisma chưa hỗ trợ GENERATED ALWAYS AS ... STORED. Sau khi chạy prisma migrate dev lần đầu, P4 mở file migration vừa sinh ra và thêm câu ALTER TABLE tương ứng.
2. Partial index (các index có mệnh đề WHERE) như idx_res_cleanup, uq_payments_one_success_per_order: khai báo @@index của Prisma không có tham số điều kiện, phải viết CREATE INDEX ... WHERE bằng tay.
3. Kiểu vector của pgvector: khai báo bằng Unsupported("vector(1024)"), nên mọi truy vấn tìm kiếm ngữ nghĩa phải dùng \$queryRaw thay vì API thông thường của Prisma.

```
// =====
// Multi-Warehouse E-Commerce & Inventory Control System
// Topic 3 – Course ID 502045 | Prisma 7.10.0 + PostgreSQL 16
// Owner: P4 (Backend Data Lead) – file Backend/prisma/schema.prisma
// =====

generator client {
  provider = "prisma-client-js"
  previewFeatures = ["postgresqlExtensions"]
}

datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
  extensions = [pgcrypto, vector, pg_trgm]
}

// ----- ENUMS -----
enum UserRole { CUSTOMER WAREHOUSE STAFF ADMIN }
enum OrderStatus { PENDING PAYMENT CONFIRMED SHIPPING DELIVERED CANCELLED }
enum ReservationStatus { HOLDING COMMITTED RELEASED EXPIRED }
enum PaymentStatus { PENDING SUCCESS FAILED }
enum PaymentMethod { MOCK_CARD MOCK_EWALLET COD }
enum AlertLevel { WARNING CRITICAL }
enum AlertStatus { OPEN RESOLVED }
enum InventoryChangeReason { RESERVE COMMIT RELEASE EXPIRE RESTOCK MANUAL_ADJUST }
enum ChatRole { USER ASSISTANT SYSTEM }
enum CartStatus { ACTIVE CONVERTED ABANDONED }

// ===== A. IDENTITY & ACCESS =====
model User {
  id          String @id @default(dbgenerated("gen_random_uuid()")) @db.Uuid
  email       String @unique @db.VarChar(255)
  passwordHash String @map("password_hash") @db.VarChar(255)
  fullName    String @map("full_name") @db.VarChar(150)
  phone       String? @db.VarChar(20)
  role        UserRole @default(CUSTOMER)
  isActive    Boolean @default(true) @map("is_active")
  createdAt   DateTime @default(now()) @map("created_at") @db.Timestamptz
  updatedAt   DateTime @updatedAt @map("updated_at") @db.Timestamptz

  refreshTokens RefreshToken[]
  carts          Cart[]
  orders         Order[]
  auditLogs      InventoryAuditLog[]
  statusChanges  OrderStatusHistory[]
  resolvedAlerts StockAlert[]
  chatSessions   ChatSession[]

  @@index([role])
  @@map("users")
}

model RefreshToken {
  id          String @id @default(dbgenerated("gen_random_uuid()")) @db.Uuid
  userId      String @map("user_id") @db.Uuid
  tokenHash   String @unique @map("token_hash") @db.VarChar(255)
  expiresAt   DateTime @map("expires_at") @db.Timestamptz
  revokedAt   DateTime? @map("revoked_at") @db.Timestamptz
  createdAt   DateTime @default(now()) @map("created_at") @db.Timestamptz

  user User @relation(fields: [userId], references: [id], onDelete: Cascade)

  @@index([userId])
  @@map("refresh_tokens")
}
```

```

}

// ===== B. CATALOG =====
model Category {
  id String @id @default(dbgenerated("gen_random_uuid()")) @db.Uuid
  name String @db.VarChar(150)
  slug String @unique @db.VarChar(160)
  parentId String? @map("parent_id") @db.Uuid
  isActive Boolean @default(true) @map("is_active")
  createdAt DateTime @default(now()) @map("created_at") @db.Timestamptz
  updatedAt DateTime @updatedAt @map("updated_at") @db.Timestamptz

  parent Category? @relation("CategoryTree", fields: [parentId], references: [id], onDelete: SetNull)
  children Category[] @relation("CategoryTree")
  products Product[]

  @@index([parentId])
  @@map("categories")
}

model Product {
  id String @id @default(dbgenerated("gen_random_uuid()")) @db.Uuid
  sku String @unique @db.VarChar(64)
  name String @db.VarChar(255)
  slug String @unique @db.VarChar(280)
  description String? @db.Text
  basePrice Decimal @map("base_price") @db.Decimal(14, 2)
  discountPercent Decimal @default(0) @map("discount_percent") @db.Decimal(5, 2)
  categoryId String? @map("category_id") @db.Uuid
  imageUrl String? @map("image_url") @db.VarChar(500)
  isActive Boolean @default(true) @map("is_active")
  createdAt DateTime @default(now()) @map("created_at") @db.Timestamptz
  updatedAt DateTime @updatedAt @map("updated_at") @db.Timestamptz

  category Category? @relation(fields: [categoryId], references: [id], onDelete: SetNull)
  inventory Inventory[]
  cartItems CartItem[]
  orderItems OrderItem[]

  @@index([categoryId])
  @@index([basePrice])
  @@map("products")
}

// ===== C. WAREHOUSE & INVENTORY (LÖI) =====
model Warehouse {
  id String @id @default(dbgenerated("gen_random_uuid()")) @db.Uuid
  code String @unique @db.VarChar(32)
  name String @db.VarChar(150)
  region String @db.VarChar(100)
  address String? @db.VarChar(500)
  capacity Int?
  defaultLowStockThreshold Int @default(10) @map("default_low_stock_threshold")
  isActive Boolean @default(true) @map("is_active")
  createdAt DateTime @default(now()) @map("created_at") @db.Timestamptz
  updatedAt DateTime @updatedAt @map("updated_at") @db.Timestamptz

  inventory Inventory[]
  orderItems OrderItem[]

  @@index([region])
  @@map("warehouses")
}

/// BẢNG TRUNG TÂM – hiện thực quan hệ N-N products <-> warehouses.
/// id dùng BigInt (BIGSERIAL) thay UUID để sắp thứ tự khóa chống deadlock.
model Inventory {
  id BigInt @id @default(autoincrement())
  productId String @map("product_id") @db.Uuid
  warehouseId String @map("warehouse_id") @db.Uuid
  stock Int @default(0)
  reservedStock Int @default(0) @map("reserved_stock")
  /// Cột GENERATED ALWAYS AS (stock - reserved_stock) STORED.
  /// Prisma không tạo được cột sinh -> khai báo Unsupported và thêm bằng migration SQL tay.
  availableStock Int @map("available_stock")
  threshold Int?
  version Int @default(0)
  lastRestockedAt DateTime? @map("last_restocked_at") @db.Timestamptz
  createdAt DateTime @default(now()) @map("created_at") @db.Timestamptz
  updatedAt DateTime @updatedAt @map("updated_at") @db.Timestamptz

  product Product @relation(fields: [productId], references: [id], onDelete: Cascade)
  warehouse Warehouse @relation(fields: [warehouseId], references: [id], onDelete: Restrict)
  reservations StockReservation[]
  auditLogs InventoryAuditLog[]
  alerts StockAlert[]
  orderItems OrderItem[]

  @@unique([productId, warehouseId], name: "uq_inventory_product_warehouse")
  @@index([productId])
  @@index([warehouseId])
  @@index([warehouseId, availableStock])
  @@map("inventory")
}

```

```

}

model InventoryAuditLog {
    id          BigInt          @id @default(autoincrement())
    inventoryId BigInt          @map("inventory_id")
    reason      InventoryChangeReason
    stockBefore Int             @map("stock_before")
    stockAfter  Int             @map("stock_after")
    reservedBefore Int          @map("reserved_before")
    reservedAfter Int           @map("reserved_after")
    quantityDelta Int           @map("quantity_delta")
    referenceType String?       @map("reference_type") @db.VarChar(40)
    referenceId String?         @map("reference_id") @db.VarChar(64)
    actorId     String?         @map("actor_id") @db.Uuid
    note        String?         @db.VarChar(500)
    createdAt   DateTime        @default(now()) @map("created_at") @db.Timestamptz

    inventory Inventory @relation(fields: [inventoryId], references: [id], onDelete: Cascade)
    actor      User?    @relation(fields: [actorId], references: [id], onDelete: SetNull)

    @@index([inventoryId, createdAt])
    @@index([referenceType, referenceId])
    @map("inventory_audit_log")
}

// ===== D. CART (tùy chọn) =====
model Cart {
    id          String          @id @default(dbgenerated("gen_random_uuid()")) @db.Uuid
    userId      String          @map("user_id") @db.Uuid
    status      CartStatus      @default(ACTIVE)
    createdAt   DateTime        @default(now()) @map("created_at") @db.Timestamptz
    updatedAt   DateTime        @updatedAt @map("updated_at") @db.Timestamptz

    user User @relation(fields: [userId], references: [id], onDelete: Cascade)
    items CartItem[]

    @@index([userId])
    @map("carts")
}

model CartItem {
    id          String          @id @default(dbgenerated("gen_random_uuid()")) @db.Uuid
    cartId      String          @map("cart_id") @db.Uuid
    productId   String          @map("product_id") @db.Uuid
    quantity    Int             @default(1)
    addedAt     DateTime        @default(now()) @map("added_at") @db.Timestamptz
    updatedAt   DateTime        @updatedAt @map("updated_at") @db.Timestamptz

    cart Cart @relation(fields: [cartId], references: [id], onDelete: Cascade)
    product Product @relation(fields: [productId], references: [id], onDelete: Cascade)

    @@unique([cartId, productId])
    @map("cart_items")
}

// ===== E. ORDER & RESERVATION (LỐI) =====
model Order {
    id          String          @id @default(dbgenerated("gen_random_uuid()")) @db.Uuid
    orderCode   String          @unique @map("order_code") @db.VarChar(32)
    userId      String          @map("user_id") @db.Uuid
    status      OrderStatus     @default(PENDING_PAYMENT)
    subtotal    Decimal         @default(0) @db.Decimal(14, 2)
    discountTotal Decimal       @default(0) @map("discount_total") @db.Decimal(14, 2)
    shippingFee Decimal         @default(0) @map("shipping_fee") @db.Decimal(14, 2)
    grandTotal  Decimal         @default(0) @map("grand_total") @db.Decimal(14, 2)
    currency    String          @default("VND") @db.Char(3)
    shippingRecipient String    @map("shipping_recipient") @db.VarChar(150)
    shippingPhone String        @map("shipping_phone") @db.VarChar(20)
    shippingAddress String      @map("shipping_address") @db.VarChar(500)
    shippingRegion String?      @map("shipping_region") @db.VarChar(100)
    reservationExpiresAt DateTime? @map("reservation_expires_at") @db.Timestamptz
    cancelledReason String?     @map("cancelled_reason") @db.VarChar(255)
    confirmedAt DateTime?       @map("confirmed_at") @db.Timestamptz
    cancelledAt DateTime?       @map("cancelled_at") @db.Timestamptz
    createdAt   DateTime        @default(now()) @map("created_at") @db.Timestamptz
    updatedAt   DateTime        @updatedAt @map("updated_at") @db.Timestamptz

    user User @relation(fields: [userId], references: [id], onDelete: Restrict)
    items OrderItem[]
    reservations StockReservation[]
    payments Payment[]
    statusHistory OrderStatusHistory[]

    @@index([userId, createdAt])
    @@index([status, createdAt])
    @map("orders")
}

model OrderItem {
    id          String          @id @default(dbgenerated("gen_random_uuid()")) @db.Uuid
    orderId     String          @map("order_id") @db.Uuid
    productId   String          @map("product_id") @db.Uuid

```

```

warehouseId String @map("warehouse_id") @db.Uuid
inventoryId BigInt @map("inventory_id")
productName String @map("product_name") @db.VarChar(255)
sku String @db.VarChar(64)
unitPrice Decimal @map("unit_price") @db.Decimal(14, 2)
discountAmount Decimal @default(0) @map("discount_amount") @db.Decimal(14, 2)
quantity Int
lineTotal Decimal @map("line_total") @db.Decimal(14, 2)
createdAt DateTime @default(now()) @map("created_at") @db.Timestamptz

order Order @relation(fields: [orderId], references: [id], onDelete: Cascade)
product Product @relation(fields: [productId], references: [id], onDelete: Restrict)
warehouse Warehouse @relation(fields: [warehouseId], references: [id], onDelete: Restrict)
inventory Inventory @relation(fields: [inventoryId], references: [id], onDelete: Restrict)
reservations StockReservation[]

@@index([orderId])
@@index([productId])
@@map("order_items")
}

/// BẢNG CHỐNG OVERSELL – trái tim kỹ thuật của đồ án.
model StockReservation {
    id BigInt @id @default(autoincrement())
    orderId String @map("order_id") @db.Uuid
    orderItemId String? @map("order_item_id") @db.Uuid
    inventoryId BigInt @map("inventory_id")
    quantity Int
    status ReservationStatus @default(HOLDING)
    expiresAt DateTime @map("expires_at") @db.Timestamptz
    committedAt DateTime? @map("committed_at") @db.Timestamptz
    releasedAt DateTime? @map("released_at") @db.Timestamptz
    createdAt DateTime @default(now()) @map("created_at") @db.Timestamptz
    updatedAt DateTime @map("updated_at") @db.Timestamptz

    order Order @relation(fields: [orderId], references: [id], onDelete: Cascade)
    orderItem OrderItem? @relation(fields: [orderItemId], references: [id], onDelete: Cascade)
    inventory Inventory @relation(fields: [inventoryId], references: [id], onDelete: Restrict)

    @@index([expiresAt])
    @@index([orderId])
    @@index([inventoryId, status])
    @@map("stock_reservations")
}

model OrderStatusHistory {
    id BigInt @id @default(autoincrement())
    orderId String @map("order_id") @db.Uuid
    fromStatus OrderStatus? @map("from_status")
    toStatus OrderStatus @map("to_status")
    actorId String? @map("actor_id") @db.Uuid
    reason String? @db.VarChar(255)
    createdAt DateTime @default(now()) @map("created_at") @db.Timestamptz

    order Order @relation(fields: [orderId], references: [id], onDelete: Cascade)
    actor User? @relation(fields: [actorId], references: [id], onDelete: SetNull)

    @@index([orderId, createdAt])
    @@map("order_status_history")
}

// ===== F. PAYMENT =====
model Payment {
    id String @id @default(dbgenerated("gen_random_uuid()")) @db.Uuid
    orderId String @map("order_id") @db.Uuid
    /// KHÓA IDEMPOTENCY UNIQUE chặn callback lặp ở tầng CSDL
    transactionRef String @unique @map("transaction_ref") @db.VarChar(64)
    amount Decimal @db.Decimal(14, 2)
    currency String @default("VND") @db.Char(3)
    method PaymentMethod
    status PaymentStatus @default(PENDING)
    failureReason String? @map("failure_reason") @db.VarChar(255)
    gatewayResponse Json? @map("gateway_response")
    callbackCount Int @default(0) @map("callback_count")
    processedAt DateTime? @map("processed_at") @db.Timestamptz
    createdAt DateTime @default(now()) @map("created_at") @db.Timestamptz
    updatedAt DateTime @map("updated_at") @db.Timestamptz

    order Order @relation(fields: [orderId], references: [id], onDelete: Restrict)

    @@index([orderId])
    @@index([status])
    @@map("payments")
}

// ===== G. NOTIFICATION =====
model StockAlert {
    id BigInt @id @default(autoincrement())
    inventoryId BigInt @map("inventory_id")
    alertLevel AlertLevel @default(WARNING) @map("alert_level")
    status AlertStatus @default(OPEN)
    stockAtTrigger Int @map("stock_at_trigger")
    availableAtTrigger Int @map("available_at_trigger")

```

```

thresholdAtTrigger Int          @map("threshold_at_trigger")
triggeredAt         DateTime    @default(now()) @map("triggered_at") @db.Timestampz
resolvedAt          DateTime?   @map("resolved_at") @db.Timestampz
resolvedBy          String?     @map("resolved_by") @db.Uuid

inventory Inventory @relation(fields: [inventoryId], references: [id], onDelete: Cascade)
resolver User?     @relation(fields: [resolvedBy], references: [id], onDelete: SetNull)

@@index([triggeredAt])
@@map("stock_alerts")
}

// ===== H. AI CHATBOT (RAG) =====
model KbDocument {
  id          String   @id @default(dbgenerated("gen_random_uuid()")) @db.Uuid
  title       String   @db.VarChar(255)
  docType     String   @map("doc_type") @db.VarChar(50)
  chunkIndex  Int      @default(0) @map("chunk_index")
  content     String   @db.Text
  /// Prisma chưa hỗ trợ kiểu vector -> dùng Unsupported, truy vấn bằng $queryRaw
  embedding   Unsupported("vector(1024)")?
  tokenCount  Int?     @map("token_count")
  sourceUrl   String?  @map("source_url") @db.VarChar(500)
  isActive    Boolean  @default(true) @map("is_active")
  createdAt   DateTime @default(now()) @map("created_at") @db.Timestampz
  updatedAt   DateTime @map("updated_at") @db.Timestampz

  @@index([docType])
  @@map("kb_documents")
}

model ChatSession {
  id          String   @id @default(dbgenerated("gen_random_uuid()")) @db.Uuid
  userId      String?  @map("user_id") @db.Uuid
  sessionToken String   @unique @map("session_token") @db.VarChar(100)
  escalatedToHuman Boolean @default(false) @map("escalated_to_human")
  messageCount Int      @default(0) @map("message_count")
  startedAt   DateTime @default(now()) @map("started_at") @db.Timestampz
  endedAt     DateTime? @map("ended_at") @db.Timestampz

  user User? @relation(fields: [userId], references: [id], onDelete: SetNull)
  messages ChatMessage[]

  @@index([userId, startedAt])
  @@map("chat_sessions")
}

model ChatMessage {
  id          BigInt @id @default(autoincrement())
  sessionId   String @map("session_id") @db.Uuid
  role        ChatRole
  content     String @db.Text
  retrievedContext Json? @map("retrieved_context")
  toolCalls   Json? @map("tool_calls")
  isFallback  Boolean @default(false) @map("is_fallback")
  latencyMs   Int?   @map("latency_ms")
  tokenUsage  Int?   @map("token_usage")
  createdAt   DateTime @default(now()) @map("created_at") @db.Timestampz

  session ChatSession @relation(fields: [sessionId], references: [id], onDelete: Cascade)

  @@index([sessionId, createdAt])
  @@map("chat_messages")
}

```

12. MA TRẬN TRUY VẾT YÊU CẦU → BẢNG DỮ LIỆU

Bảng này đáp ứng checklist mục 8 của bản đặc tả (“ERD được đối chiếu với các FR liên quan”) và yêu cầu CON-04 về khả năng truy ngược code/test về Req ID.

Mã yêu cầu	Nội dung	Bảng dữ liệu liên quan	Người phụ trách
FR-AUTH-01 / 02	Đăng ký, đăng nhập, phân quyền RBAC	users, refresh_tokens	P4
FR-CAT-01 / 02 / 03	Tìm kiếm, chi tiết, gợi ý sản phẩm thay thế	products, categories, inventory	P1 / P4
FR-CART-01 / 02	Giỏ hàng và luồng checkout	carts, cart_items	P1
FR-CART-03	Pricing Engine tính tổng tiền	products.base_price, order_items (snapshot giá)	P3
FR-ORD-01	Tạo đơn hàng	orders, order_items	P3
FR-ORD-02	Stock Reservation Engine chống oversell	inventory, stock_reservations	P3 / P4
FR-ORD-03	Khách theo dõi trạng thái đơn	orders, order_status_history	P3
FR-ORD-04	Giải phóng reservation hết hạn	stock_reservations, inventory, orders	P3
FR-ORD-05	Nhân viên cập nhật trạng thái giao hàng	order_status_history, orders	P3 / P2
FR-PAY-01 / 02	Thanh toán mô phỏng, xử lý idempotent	payments (transaction_ref UNIQUE)	P3
FR-INV-01 / 02	Quản lý và đồng bộ tồn kho đa kho	inventory, warehouses, inventory_audit_log	P4
FR-DASH-01 / 02	Dashboard tồn kho và báo cáo vận hành	inventory, orders, order_items, stock_alerts	P2
FR-NOT-01	Cảnh báo tồn kho thấp thời gian thực	stock_alerts, inventory.threshold	P5
FR-BOT-01 / 02	Truy xuất ngữ cảnh và sinh câu trả lời	kb_documents (pgvector), chat_messages	P5
FR-BOT-03	Function calling tra cứu đơn / tồn kho	orders, inventory (chỉ đọc), chat_messages.tool_calls	P5
FR-BOT-04 / 05	Fallback CSKH và giao diện chat widget	chat_sessions.escalated_to_human, chat_messages.is_fallback	P5 / P1
FR-ADM-01 / 02	Quản trị sản phẩm, giá và kho	products, categories, warehouses	P4 / P2
NFR-03.1 / 03.2	Toàn vẹn dữ liệu và giao dịch ACID	inventory_audit_log, các CHECK constraint	P3 / P4
NFR-04.1 / 04.2	Không oversell, thanh toán idempotent	CHECK reserved_stock ≤ stock, UNIQUE transaction_ref	P3 / P4
NFR-07.2	Đánh giá chất lượng chatbot	chat_messages.retrieved_context, is_fallback, latency_ms	P5

13. NHỮNG ĐIỂM NHÓM CẦN CHỐT TRƯỚC KHI CODE

Bản đặc tả mục 9 liệt kê các điểm còn để ngỏ. Dưới đây là những điểm có ảnh hưởng trực tiếp tới cơ sở dữ liệu, kèm cách thiết kế hiện tại đang xử lý tạm và mức độ tổn kém nếu đổi ý về sau.

Điểm cần chốt	Thiết kế đang xử lý thế nào	Chi phí nếu đổi ý sau	Hạn chốt
Nhà cung cấp embedding cho RAG	Đang để vector(1024) theo Voyage AI. Claude không có API embedding.	RẤT CAO — đổi số chiều sau khi đã nạp dữ liệu buộc phải tạo lại bảng và vector hóa lại toàn bộ.	Sprint 1
Giỏ hàng client-side hay server-side	Đã thiết kế sẵn carts và cart_items, đánh dấu tùy chọn.	THẤP — nếu chọn client-side chỉ cần xóa hai bảng, không bảng nào khác phụ thuộc.	Sprint 1
Ngưỡng low-stock so với stock hay available_stock	Khuyến nghị so với available_stock vì đó mới là số hàng thực sự bán được.	THẤP — chỉ đổi biểu thức trong service, không đổi schema.	Sprint 1
Thuật toán chọn kho khi checkout	Schema đã hỗ trợ mọi thuật toán: order_items mang warehouse_id riêng từng dòng.	THẤP — hoàn toàn nằm ở tầng service.	Sprint 1
Quy tắc phí vận chuyển và thuế	Đã có sẵn cột shipping_fee trên orders, mặc định 0.	TRUNG BÌNH — nếu tính phí theo từng dòng hàng thì phải thêm cột vào order_items.	Sprint 2
TTL reservation 15 phút	Đặt làm giá trị mặc định của tham số, lưu tuyệt đối vào expires_at.	THẤP — là tham số cấu hình, không phải hằng số trong schema.	Sprint 1
Quyền cập nhật tồn kho của WAREHOUSE_STAFF	Schema không phân biệt; kiểm soát bằng RolesGuard ở tầng controller.	THẤP — thuần túy tầng ứng dụng.	Sprint 1
Cập nhật trạng thái đơn bằng polling hay WebSocket	Bảng order_status_history phục vụ được cả hai cách.	THẤP — không ảnh hưởng schema.	Sprint 2

13.1. Việc cần làm tiếp theo

- P4 đưa schema Prisma vào Backend/prisma/schema.prisma, chạy migrate rồi bổ sung ba hạng mục ở mục 11 bằng migration SQL tay.
- P4 viết Seed.ts theo dữ liệu chuẩn mà bản đặc tả mục 6.1 quy định: 3 kho, 200 sản phẩm, 600 dòng tồn kho, 100 khách hàng, 2 admin, 2 nhân viên kho, 30 đoạn tài liệu chính sách.
- P4 và P3 chốt chữ ký hàm lockAndCheckStock trước khi P3 viết StockReservationService — tham khảo hàm mẫu ở mục 10.2.
- P5 chốt nhà cung cấp embedding và xác nhận lại số chiều vector trong schema.
- Cả nhóm rà soát chín xung đột ở mục 3 và xác nhận bằng văn bản, vì tên cột sẽ đi thẳng vào API contract của mọi người.

— HẾT —